2차 이용 권리 관리 언어 마스터 규격

항목	내용
문서명	2차 이용 권리 관리 언어 마스터 규격 (영문 가칭 **MDRL**, Music Derivative Rights Language [확인 필요])
버전	v0.2 (초안)
작성일	2026-09-29
문서 지위	음악의 2차 이용 및 n차 창작에서, 창작자·실연자·참여자·권리자·플랫폼이 공유하는 **권리 관리 언어의 표준 규격(마스터)**. 공통 코어·층 분리·불변식·확장점을 정한다. 특정 제품·특정 창작 모드의 어휘는 프로파일이 등록한다.
적용 범위	1차 음악 저작권·저작인접권 위에서 성립하는 2차 이용과 n차 재이용의 (i) 권리 사실 기록, (ii) 이용허락 판정, (iii) 금액 안분, (iv) 정산 원장 접점. 법적 저작물 성립·귀속 판정, 요율 협상, 세무, 지급 실행은 범위 밖이다(4.2절).
용어 관례	**필수**(MUST): "~해야 한다", "~해서는 안 된다"(MUST NOT). **권고**(SHOULD): "~하는 것이 좋다". **선택**(MAY): "~할 수 있다". 규범 문장은 **[N-번호]** 로 식별한다. 본문의 비율·금액·식별자는 모두 **예시 값**이며 계약 값·기본값이 아니다.
버전 정책	마스터는 `v0.x` 초안 후 `v1.0`을 발행한다. `v1.0`은 서로 다른 창작 모드의 프로파일 두 개가 같은 코어를 공유함을 확인한 뒤에 발행한다. 게시한 `(module, id, version)` 문서는 불변이며, 수정은 새 `version`만 허용한다(4.5절). 프로파일은 대상 마스터 버전을 선언해야 한다.
우선 순위	① 본 마스터 규격의 코어 불변식 > ② 프로파일 규격의 어휘·인코딩 > ③ 구현. 코어와 프로파일이 충돌하면 코어가 우선하고, 어휘·제품 결정이 충돌하면 프로파일이 우선한다(4.4절).
성숙도 표기	**확정**: 규범 문장과 불변식이 고정됨. **방향 확정(문법 미고정)**: 책임·의미론은 확정, 표면 문법·IR 스키마는 이후 버전.

목차

1. 음악산업의 권리 관리 현황 및 이슈
2. 다양한 n차 창작에서의 권리관리 문제점
3. 2차 이용 권리 관리 언어 규격의 필요성
4. 설계 목표 및 방향
5. 핵심 개념 (특허적 요소)
6. DSL 구조 및 설계
7. 시스템 구현
8. 응용분야 및 권리 문제 해소 효과

- 부록 A. 용어집
- 부록 B. 최소 참조
- 부록 C. 외부 출처
- 부록 D. 변경 이력

1. 음악산업의 권리 관리 현황 및 이슈

이 장은 본 규격이 전제로 삼는 **현행 권리 관리 체계**와, 그 체계가 반복적으로 드러내는 한계를 정리한다. 수치는 1차 출처에서 확인된 것만 쓰고, 확인되지 않은 것은 정성적으로 기술한다(부록 C).

1.1 1차 권리의 겹침

한 곡의 음원에는 서로 다른 법적 근거를 가진 권리가 동시에 존재한다. 국내 저작권법 기준으로 다음 세 층으로 나눈다.

층	권리	주체	현행 관리 방식
저작권(작품)	저작재산권(복제·공연·공중송신·배포·2차적저작물작성 등), 저작인격권(성명표시·동일성유지)	작사·작곡·편곡자, 음악출판사	신탁곡은 집중관리단체가 재산권을 관리한다
저작인접권(실연)	실연자의 복제·배포·전송권, 방송·디지털음성송신 보상청구권, 실연자 인격권	주실연자, 부실연자(코러스·세션)	부실연자는 일괄 양도 관행과 별개로 인접권이 존속한다
저작인접권(음반)	음반제작자의 복제·배포·전송권, 보상청구권	음반제작사·레이블(마스터 권리자)	멀티트랙 파일은 마스터의 구성요소다

2차적저작물에 대해서는 저작권법 제5조가 "원저작물을 번역·편곡·변형·각색·영상제작 그 밖의 방법으로 작성한 창작물"을 독자적 저작물로 보호하되 원저작자의 권리에 영향을 미치지 않는다고 정하고, 제22조가 2차적저작물작성권을 원저작자의 권리로 둔다. 제46조는 이용허락을 받은 자가 "허락받은 이용 방법 및 조건의 범위 안에서" 이용하도록 하고, 이용권 양도에도 동의를 요구한다(출처: 국가법령정보센터, 부록 C).

이 구조에서 나오는 전제 세 가지는 본 규격 전체에 적용된다.

1. **하나의 결과물 = 여러 권리 층의 동시 이용.** 보컬 트랙 하나를 바꾸거나 더하는 행위도 작품(2차적 이용 가능성), 원 녹음(마스터), 원 실연, 신규 실연의 허락을 각각 건드린다.
2. **허락은 범위 안에서만 유효하다.** 행위·지역·매체·기간의 교집합을 벗어난 이용은 별도 허락이 필요하다.
3. **2차적저작물의 성립은 법적 판단이다.** 파일이 만들어졌다는 사실이 새 저작권을 만들지 않는다. 본 규격은 이용 사실과 계약상 대가를 기록·계산하며, 저작물 성립·귀속을 판정하지 않는다.

1.2 집중관리(신탁관리) 체계

국내에서 저작권신탁관리업은 저작권법 제105조에 따라 문화체육관광부장관의 허가를 받아야 하며, 신탁관리업자가 이용자로부터 받는 사용료의 요율·금액은 장관의 승인을 받아 정한다(제105조 제9항). 한국저작권위원회가 공개하는 신탁관리단체 규정 목록에는 음악 분야에서 **한국음악저작권협회(KOMCA)**, **함께하는음악저작인협회(KOSCAP, 함저협)**, **한국음악실연자연합회(FKMP, 음실련)**, **한국음반산업협회(RIAK)** 가 포함되어 있다. 저작권비즈니스지원센터의 권리별 단체 안내는 음악의 전송권을 위 4개 단체가, 공연권을 KOMCA·KOSCAP·FKMP 및 보상금수령단체가 다룬다고 표시한다(부록 C).

해외에서는 작품의 공연권·기계적복제권을 관리하는 PRO/MRO/CMO(예: ASCAP·BMI·PRS·GEMA·JASRAC)와, 미국의 디지털 기계적복제권 포괄 라이선스를 관리하는 The MLC, 디지털 실연 보상금을 분배하는 SoundExchange가 있다. The MLC는 자신의 범위가 미국 적격 디지털 서비스에 대한 포괄 기계적 라이선스이며 공연·동기화·음반 로열티를 다루지 않는다고 명시한다. SoundExchange가 설명하는 미국 법정 디지털 실연 보상금 분배 구조는 음원 권리자 50%, 주실연자 45%, 비주실연자 기금 5%다(출처: 각 기관 공식 페이지, 부록 C. 이 비율은 특정 제도의 사례이며 본 규격의 기본값이 아니다).

집중관리 체계가 본 규격에 주는 함의는 다음과 같다.

- 권리자와 **수령자가 다를 수 있다.** 저작자의 몫을 단체가 수령·분배한다. 이 관계는 권리 사실이 아니라 **위임(mandate)** 사실이다.
- **권리 종류·지역·기간별로 관리 주체가 다르다.** 하나의 불리언 플래그(신탁 여부)로는 표현되지 않는다.
- 단체 정산은 **보고 → 매칭 → 분배 주기**를 가지며, 미매칭·지연 보고는 정상 운영 상태로 존재한다.
- 단체 요율·배분 규정은 **고정된 이용 유형과 고정된 청구권자 집합**을 전제한다. n차 결과물마다 재료·역할·허락 범위가 달라지는 이용은 그 전제 밖에 있다.

1.3 식별자와 메타데이터 표준

식별자/표준	대상	관리	확인 출처
ISWC (ISO 15707:2022)	음악 작품(무형의 창작물). 녹음·악보 등 실현물은 대상이 아님	집중관리단체 계열	ISO
ISRC (ISO 3901:2019)	음원·뮤직비디오 녹음. 상품·묶음 번호 부여에는 쓰지 않음	IFPI가 국제 등록기관, 국가별 에이전시	ISO, isrc.ifpi.org
ISNI	창작자·법인 등 공적 정체성	ISNI 등록 에이전시	2차 자료(부록 C)
IPI	권리 단체 체계의 이해관계자(작사·작곡·출판사) 번호	권리 단체 체계	2차 자료(부록 C)
CWR	출판사↔단체 간 작품 등록·개정 파일 표준	CISAC	CISAC, The MLC
DDEX ERN	발매·리소스(녹음)·거래 조건 통지(레이블/유통사→DSP)	DDEX	DDEX KB

식별자/표준	대상	관리	확인 출처
DDEX DSR	DSP→권리자 판매·이용 보고	DDEX	DDEX KB
DDEX RIN	녹음 세션 기여자·역할·악기·구성요소 정보	DDEX	DDEX KB
DDEX MEAD	마케팅용 부가 메타데이터	DDEX	DDEX KB

이 표준들은 **사실과 거래를 운반**한다. 어느 것도 다음을 판정하지 않는다.

- 결과물 하나에 참여한 구성요소의 사용량과 계약 정책으로 금액을 어떻게 나누는가.
- 이 조합을 이 채널·지역·기간에 유통해도 되는가.
- 이용 후에 등록된 권리 이전을 어느 시점 기준으로 반영하는가.
- 분쟁 중인 지분 그룹을 어떻게 보류하는가.

ISWC는 고정된 작품을, ISRC는 고정된 녹음을 가리킨다. n차 결과물은 여러 작품·여러 녹음·여러 실연의 **사용 관계**이며, 그 관계 자체에 부여된 산업 식별자는 없다. CWR은 작품 지분 등록에 한정되고, DDEX는 공급망 메시지다. 본 규격은 이 표준들을 **어댑터 경계**로 두고 대체하지 않는다(6.6절).

1.4 계약·약관·정산 스크립트

현행 거래의 대부분은 다음 세 형태로 닫힌다.

형태	강점	n차 창작에서 부족한 점
개별 계약서(이용허락·공동저작·실연·마스터)	당사자 합의의 법적 정본	자연어. 깊이별 승계·교집합·금액 보존을 기계적으로 평가할 수 없다. 개정본과 적용 시점의 대응이 수작업이다
플랫폼 이용약관·포괄 라이선스	대량 UGC를 일률 처리할 수 있다	결과물별 재료·범위·의무의 차이를 잃는다. 약관 범위가 외부 유통·상업 이용·재이용까지 승계되는지가 불명확한 경우가 많다
정산 스크립트(내부 코드·스프레드시트)	금액을 계산한다	절차적이다. `now()`, 부동소수, 외부 조회에 의존하기 쉽다. 제3자가 같은 입력으로 같은 결과를 재현할 정규 입력이 없다

계약은 **한 거래의 조건**을 정하고, 약관은 **한 플랫폼의 일률 범위**를 정하고, 스크립트는 **한 운영자의 계산 절차**를 정한다. n차 창작은 거래·플랫폼·운영자가 바뀌어도 같은 재료 사실이 여러 결과물에 재사용되므로, 세 형태 모두 **재사용 가능한 규칙의 정본**이 되지 못한다.

1.5 기존 권리 표현 언어

언어	성격	강점	n차 이용 안분에 부족한 점
ODRL 2.2 (W3C Recommendation, 2018)	Policy = Permission / Prohibition / Duty + Constraint. Set·Offer·Agreement	허락·금지·의무의 표현과 평가 의미론이 표준화됨	금액 안분·지분 벡터·이중 시간축·반올림·금액 보존·결과 digest가 없다. "허용되는가"에 답하고 "얼마를 누구에게"에는 답하지 않는다
ccREL (Creative Commons, W3C Member Submission 2008)	RDF 기반 라이선스 메타데이터	웹 문서에 라이선스 조건을 기계 가독으로 부착	공개 라이선스 조건 표시가 목적. 다당사자 지분·계약·정산 개념이 없다
MPEG-21 REL (ISO/IEC 21000-5)	XrML 계열 XML 권리 표현, DRM 지향	폐쇄 시스템에서의 허락 발급·검증	권리 사실의 시간 변화·분쟁·안분·감사 재계산을 다루지 않는다

공통점은 **"허락 표현" 또는 "사실 운반"** 중 하나에 특화되어 있고, **n차 결과물에 대한 결정적 금액 계산과 검증 가능한 공개**를 언어 수준에서 다루는 것이 없다는 점이다. 본 규격은 ODRL을 대체하지 않고, 이용허락 층의 출력을 ODRL Policy/Permission/Duty로 내보내는 **단방향 어댑터**를 허용한다. 손실 항목(안분·시간축·digest)은 어댑터 명세에 명시해야 한다.

1.6 알려진 구조적 문제

1.6.1 메타데이터 결손과 미매칭 로열티

이용 보고의 녹음 정보가 작품·권리자와 매칭되지 않으면 금액이 지급되지 못한 채 쌓인다. 미국 사례는 규모를 공개한다. The MLC에 따르면 2021년 21개 DSP가 2007~2020년 이용분의 "historical unmatched royalties"를 이전했고, 그 총액은 최종 요율 기준 약 3억 9,700만 달러였다. The MLC는 이 중 약 3억 1,700만 달러(약 80%)를 등록 작품에 매칭했고 약 2억 2,900만 달러(약 58%)를 분배했으며, 남는 미청구 금액의 시장점유율 기반 분배(market share distribution)는 2027년에 시작할 계획이라고 밝힌다(출처: The MLC, 부록 C. 수치는 해당 페이지 게시 시점 기준).

국내 실연자 크레딧·메타데이터 결손으로 인한 미배분은 업계에서 반복 언급되나, 국내 미배분 규모의 공개 통계는 본 문서 작성 시점에 1차 출처로 확인하지 못했으므로 수치를 적지 않는다.

식별자 표준이 존재해도 **매칭이 실패하면 지급이 멈춘다.** n차 결과물은 재료가 여러 개이므로 매칭 실패 지점이 깊이에 비례해 늘어난다.

1.6.2 지분(split) 분쟁

작품 지분은 공동 창작자 간 합의로 정해지며 단체마다 등록된 지분이 다를 수 있다. 지분 합이 100%를 넘거나 부족한 상태(초과 청구, 미확인 잔여)는 어느 한쪽이 "옳다"고 시스템이 판단할 수 없고, 정산은 그 몫을 보류해야 한다. 현행 단체 등록·정산 스크립트는 초과 청구를 비례 축소하거나, 미확인분을 임의로 나누는 운영을 택하기 쉽다. 두 운영 모두 분쟁 해소 후 역분개를 복잡하게 만든다.

1.6.3 불투명한 정산과 긴 정산 주기

권리자가 받는 명세서는 통상 집계 단위(기간·서비스·국가)로 요약되며, 개별 이용 건에서 자기 금액까지의 계산 경로를 독립적으로 재계산할 수단이 없다. 정산 주기는 보고→매칭→분배 단계를 거치므로 이용 시점과 수령 시점 사이에 지연이 생기고, 그 사이 권리 사실(지분·소유권·위임)이 바뀔 수 있다. 정량적 평균 지연은 서비스·단체·국가마다 달라 여기서 수치를 제시하지 않는다.

1.6.4 UGC·리믹스의 권리 충돌

팬 리믹스·커버·매시업은 원 작품의 2차적 이용, 원 녹음의 복제·변형, 원 실연의 이용을 동시에 수반한다. 플랫폼별로 "허용 범위"가 약관·포괄 계약으로 정해지지만, 그 범위가 결과물의 유통 채널(플랫폼 내/외부, 지역, 상업 이용)까지 승계되는지가 불명확한 경우가 많다. 약관은 일률이고 계약서는 건별이어서, **재료별 범위의 교집합을 결과물마다 계산하는 층**이 없다.

1.6.5 AI 생성물의 귀속

미국 저작권청은 2025년 1월 29일 발표한 「Copyright and Artificial Intelligence, Part 2: Copyrightability」에서 인간 저작이 저작권의 전제이며, 프롬프트만으로는 출력물에 대한 충분한 통제가 인정되지 않고, 인간이 AI 출력물을 선택·배열·수정한 부분에 한해 보호가 성립할 수 있다고 정리했다(부록 C). IFPI는 2026년 7월 AI 활용 녹음의 공식 차트 적격성 원칙을, 2026년 9월 14일에는 스트리밍 무결성 공동 약속(SII)을 발표했다. 따라서 AI 보이스·AI 믹스가 개입한 결과물에서 "누가 무엇에 기여했고 무엇이 보호 대상인가"는 계산 규칙이 판정할 수 없는 문제이며, 시스템은 **기여 사실과 계약상 대가를 기록·계산**하는 데 그쳐야 한다.

1.6.6 부정 이용과 무결성

스트리밍 조작·도난 결제·자기거래는 사용량 기반 배분을 왜곡한다. IFPI SII는 확인(VERIFY)·검사(VET)·조치(ACT)·공유(SHARE)·측정(MEASURE)의 다섯 의무를 제시한다. 계산 규격은 부정 판정을 하지 않지만, **판정 전 격리(보류)를 결정적으로 처리할 수 있어야** 한다. 현행 정산 스크립트는 이 격리를 사후 보정으로 처리하는 경우가 많아, 이미 지급된 금액의 회수 범위가 커진다.

1.7 소결: 현행 체계가 닫지 못하는 층

1차 권리 구조는 층이 여러 개이고, 각 층의 관리 주체·수령 주체·시간축이 다르다. 현행 도구의 역할은 다음과 같이 정리된다.

현행 도구	하는 일	하지 않는 일
집중관리단체 요율·배분 규정	고정 이용 유형에 대한 일률 분배	결과물별 재료·사용량·깊이별 역할
ISWC/ISRC/ISNI/IPI	작품·녹음·주체의 식별	파생 연쇄와 사용 관계
CWR/DDEX	등록·발매·이용 보고의 운반	규칙 판정과 금액 계산 의미론
계약서·약관	당사자 합의·일률 범위	기계 평가, 깊이별 교집합, 버전 고정
ODRL/ccREL/MPEG-21 REL	허락·금지·의무	안분, 이중 시간축, 금액 보존, 결과 검증
정산 스크립트	운영자 내부의 금액 산출	선언적 정본, 제3자 재계산, 결정성 보장

n차 결과물의 사용량·지분·계약을 결정적으로 계산하고 제3자가 재계산할 수 있게 공개하는 층은 비어 있다. 2장은 이 빈 층이 n차 창작에서 어떤 구체적 문제로 나타나는지 정리한다.

---

2. 다양한 n차 창작에서의 권리관리 문제점

2.1 n차 창작의 정의와 행위자

본 규격에서 **n차 창작(n-th order creation)** 은 다음 연쇄로 정의한다.

- **1차(원저작물·원녹음)**: 작품(Work)과 그 최초 녹음(Recording). 1.1절의 세 권리 층이 성립한다.
- **2차 이용(secondary use)**: 1차 결과물의 구성요소(트랙·구간·음성 특성)를 허락 범위 안에서 재사용·재배치·추가하여 새 결과물을 만드는 행위. 예: 보컬 트랙 재배치, 보컬 트랙 추가 녹음, AI 믹스/마스터링, 커버·리믹스, 샘플링.
- **3차 이상 재이용**: 직전 결과물을 다시 재료로 쓰는 행위. 예: 2차 결과물의 재판매, 외부 유통, 재리믹스.

여기서 "이용"과 "저작물 성립"을 구분한다. 2차 **이용**은 사실(어떤 재료가 어떤 결과물에 들어갔는가)이고, 2차적**저작물**의 성립과 귀속은 법적 판단이다. 본 규격은 전자만 기록·계산하며 후자를 판정하지 않는다(4.2절).

행위자는 깊이에 따라 역할이 갈린다. 같은 사람이 여러 역할을 동시에 가질 수 있다.

행위자	전형적 기여	현행 체계에서의 위치
원저작자(작사·작곡·편곡)	작품	CWR/단체 등록, ISWC
원 실연자	원 녹음의 실연	인접권 단체, 크레딧
음반제작자	원 마스터	레이블 계약, ISRC
2차 창작자	재배치·편곡·믹스·추가 창작	건별 계약 또는 약관. 고정 식별자 없음
참여 실연자(신규 녹음)	새 실연, 경우에 따라 새 마스터	실연 계약. 원 마스터와의 관계가 문서마다 다름
AI 모델·데이터 제공자	학습·추론·합성	기존 역할 어휘에 없음. 실연·저작에 억지 매핑되기 쉬움
플랫폼	유통·결제·정산 운영	약관·내부 스크립트
팬 기여자	선택·비율 지정·UGC	이용권자. 저작권자로 오인되기 쉬움

2.2 창작 모드 스펙트럼

창작 모드	재료	새로 생기는 기여	특유의 권리 질문
보컬 트랙 재배치	원곡 멀티트랙 보컬, 고정 반주	선택과 배치	선택은 저작 기여인가, 배분 기준 입력인가
보컬 트랙 추가 녹음	원곡 반주 + 신규 실연	새 실연, 새 녹음	새 녹음의 마스터가 원 마스터를 자동 승계하는가
AI 믹스/마스터링	완성 트랙 집합	믹스 파라미터, 모델 실행	모델 제공자는 청구권자인가, 어떤 계약 대가인가
AI 보이스 실연	동의된 원천 음성 + 모델	합성 실연	원천 음성 제공자·모델 운영자의 각 허락(학습/생성 분리), 인간 실연료와의 동일시 금지
커버/리믹스	작품(+원 녹음 일부)	새 실연·편곡·믹스	작품 이용 허락 범위(편곡 포함 여부), 원 녹음 사용 여부
샘플링	원 녹음의 구간	새 작품·녹음	원 녹음과 원 작품 모두의 허락, 곡별 안분
이용자 지정 비율 묶음	권리자가 정의한 후보 집합	이용자가 지정한 비율	이용자 비율이 원저작자 몫을 바꾸지 않아야 함

어느 모드도 다른 모드의 자산 모델로 **강제 변환되어서는 안 된다**(4.3절 원칙 7). 공통으로 필요한 것은 사실·허락·안분의 코어이고, 모드별 어휘는 프로파일이 등록한다.

2.3 문제점과 현행 도구의 실패

P1. 깊이를 가로지르는 권리 연쇄 추적

3차 결과물의 청구권자를 알려면 2차 결과물의 청구권자와 그 재료인 1차의 청구권자를 모두 따라가야 한다. 결과물 수가 늘면 경로 수가 폭증한다. 경로를 결과물마다 복제하면 원 사실이 정정될 때 일관성이 깨진다.

- **ISWC/ISRC**: 각 작품·녹음을 가리킬 뿐, 사용 관계를 표현하지 않는다. 3차 결과물에 새 코드를 부여하면 1·2차와의 연결이 끊긴다.
- **CWR/DDEX**: 한 작품 또는 한 발매의 정적 크레딧을 운반한다. 깊이별 참조를 한 정본에 두고 결과물은 사용량만 들게 하는 모델이 없다.
- **계약서**: 건마다 당사자를 다시 나열한다. 원 사실 정정이 이미 서명된 하위 계약에 자동 전파되지 않는다.

P2. 깊이별 청구권자 판정

같은 사람이 1차에서는 작곡가, 2차에서는 재녹음 실연자, 3차에서는 결과물 판매자일 수 있다. 역할을 합쳐 한 금액으로 만들면 어느 권리 층의 대가인지 설명할 수 없고, 단체 보고·세무 구분이 불가능해진다.

- **단체 배분 규정**: 작품 분배와 실연 분배를 별 제도로 두지만, 한 결과물의 한 정산에서 역할별 원장 라인을 끝까지 보존하지 않는다.
- **정산 스크립트**: 당사자별 합계를 내는 구현이 많다. 역할 키 없는 합계는 사후 설명을 불가능하게 한다.

P3. 지분 폭증과 반올림

1차 지분(작곡 60/40) × 2차 사용량 비율 × 3차 계약 비율을 곱하면 유리수 분모가 커지고, 원 단위로 떨어지지 않는다. 층마다 반올림하면 합계가 상위 금액과 어긋나고, 마지막에 한 번만 반올림하면 중간 층의 금액 보존을 검증할 수 없다. 반올림 규칙이 운영 코드마다 다르면 같은 입력에 다른 결과가 나온다.

- **단체 요율표**: 고정 비율의 정수 분배를 전제하며, 깊이 곱셈의 잔여를 층마다 보존하는 규칙을 공개하지 않는다.
- **정산 스크립트**: 부동소수와 임의 반올림이 섞이기 쉽다. 제3자가 잔여 1원의 귀속을 재현할 수 없다.

P4. 허락 범위의 승계

1차 허락이 "국내 앱 내 소장·재생"이면, 그 재료를 쓴 2차 결과물을 "해외 숏폼"에 쓸 수 있는가? 상식적으로는 안 되지만, 시스템이 이를 **범위의 교집합**으로 계산하지 않으면 검색용 후보 상한(합집합)이 실제 허락으로 오용된다.

- **약관·포괄 라이선스**: 플랫폼 내부의 일률 범위를 정한다. 재료별 범위의 교집합을 결과물마다 도출하지 않는다.
- **ODRL**: Permission과 Prohibition을 나열하고 충돌 전략을 둘 수 있으나, 정책 간 범위 합성을 격자 연산(meet only)으로 고정하지 않는다.
- **계약서**: "하위 이용은 상위 범위를 넘을 수 없다"를 문장으로 쓸 수 있으나, 기계 검증되지 않는다.

P5. 시점: 이용 후에 바뀌는 권리 사실

9월 10일 이용분을 10월에 정산할 때, 9월 20일에 등록된 "9월 1일부터 유효한 권리 이전"을 반영해야 하는가? 두 답이 모두 필요하다. 당시 판단의 재현(감사)과 현재 증빙 기준의 재계산(정정) 모두 요구된다. 시간축이 하나면 둘 중 하나를 잃는다.

- **단체 등록**: 보통 현재 유효 지분만 조회한다. "그때 무엇을 알고 있었는가"를 남기지 않는다.
- **CWR 개정 파일**: 작품 지분의 개정 이력을 운반할 수 있으나, 이용 시점과 인지 시점을 한 질의의 두 축으로 고정하지 않는다.
- **정산 스크립트**: `as_of` 하나만 두거나, 실행 시각(`now`)에 의존한다.

P6. 연쇄 중간의 분쟁

2차 결과물에 들어간 재료 하나의 마스터 소유권이 다투어지면, 그 재료 몫만 보류하고 나머지는 지급해야 한다. 남은 사람들에게 보류분을 재분배(재정규화)하면 분쟁 해소 후 역분개가 복잡해지고, 분쟁 당사자 몫이 타인에게 선지급되는 결과가 된다.

- **단체 운영**: 초과 청구를 비례 축소하거나 분쟁 곡 전체를 지급 정지하는 이분법이 흔하다. 그룹 단위 보류와 타 역할 정상 지급을 한 계산으로 표현하지 않는다.
- **계약서**: 에스크로 조항은 있어도, 벡터 합 1을 유지한 채 보류 키를 남기는 계산 규칙이 없다.

P7. 기존 스키마에 없는 역할

팬 기여자, AI 모델 제공자, 플랫폼 운영 풀은 DDEX·CWR의 역할 어휘에 없다. 이를 기존 역할(예: 실연자)에 억지로 매핑하면 보고와 세무가 왜곡된다. 반대로 모든 모드의 역할을 하나의 고정 목록에 넣으면 다른 창작 모드에 불필요한 어휘가 강제된다.

P8. 안분 결과를 허락으로 오해하는 위험

"금액이 계산되었으니 이용이 정당하다"는 추론은 성립하지 않는다. 허락이 없는 이용의 매출도 계산은 가능하며, 그 계산 결과는 사고 처리 근거일 뿐 이용을 정당화하지 않는다. 현행 정산 스크립트는 허락 판정과 금액 계산을 한 함수에 섞기 쉽다. ODRL은 허락만 다루고 금액은 다루지 않아, 두 판정을 분리한 채 함께 쓰는 표준이 없다.

2.4 작동 예시 체인

가상 곡 **W001 "별빛"** 으로 네 깊이를 따라간다. 모든 값은 예시다.

```
1차  Work W001
       작곡가 A 60% / 작사가 B 40%   (B는 단체 C에 WORK 권리 위임)
     Recording R001
       마스터 L 100%
       실연: 원 보컬 P1, 고정 코러스 P2
     허락: A·B·L·P1·P2 — 국내 앱 재생. P2는 국내 앱 + 숏폼

2차  신규 실연자 P3가 보컬 트랙을 추가 녹음 → Recording R002 (제작사 N)
     P3 허락: 국내 앱만
     유효 범위 = R001 재료 범위 ⊓ R002 범위 = {국내 앱}
     해외·숏폼·광고는 불허 (P4)
     사용량 u = {P1 트랙: 60000ms, P3 트랙: 30000ms, P2: 90000ms, 반주: 90000ms}
     매출 e1: 소장권 판매 10,000원 (KRW)

3차  AI 믹스 세션 M001 (엔지니어 E, 모델 운영자 Q) → 결과물 X001
     모델 대가는 계약 벡터. 인간 실연료와 동일시하지 않음
     학습 허락과 생성 허락은 별개
```

```
4차  X001 재이용 e2: 20,000원, 계약상 거래 로열티 5% = 1,000원만 권리자 풀 대상
     10월: 9월 1일부로 유효한 "L → L2 마스터 이전"이 9월 20일 등록됨 (P5)
     11월: A의 작품 지분에 이의 제기 → WRITTEN_BY 그룹 분쟁 (P6)
```

언어가 답해야 하는 질문과, 현행 도구가 주지 못하는 답의 위치는 다음과 같다.

질문	현행 도구	본 규격의 층	답
X001을 해외 앱에 배포할 수 있는가	약관은 일률, 계약은 건별. 교집합 계산 없음	grant	`DENY`(P3 범위 밖). 안분 결과와 무관(P8)
e1의 권리자 풀 7,000원(예시)을 누가 얼마씩 받는가	스크립트마다 다름. 재현 수단 없음	allocation	정책 버전 × 투영 벡터 × 스냅샷으로 결정적 계산, 원 단위 보존
마스터 몫은 L인가 L2인가	현재 등록 조회 또는 실행 시각	facts (`as_of`, `known_at`)	`as_of=9/10, known_at=9/10` → L. `as_of=9/10, known_at=10/1` → L2. 둘 다 재현 가능. 정산에 쓸 쪽은 정책의 `bind.time`
A 분쟁 중 B와 단체 C의 몫은	곡 전체 정지 또는 비례 축소	facts + allocation	A가 속한 `WRITTEN_BY` 그룹 전체를 보류. B·C 몫도 그 그룹에 있으므로 함께 보류. 실연·마스터 버킷은 정상 지급
e2에서 원 권리자가 받는 것은	재판매 계약을 별 정산으로 다시 짬	allocation	1,000원이 같은 정책 구조로 안분. 판매자 귀속금은 정산 대상이 아님
L2·A·E·Q가 각자 검증할 수 있는가	명세서 신뢰. 재계산 불가	core (digest, 공개)	공개본 IR + 커밋먼트 + 결과 digest 대조. 당사자는 자기 몫만 복원

2.5 요구사항 도출

문제	요구사항	대응 개념(5장)
P1	사실은 한 곳에, 결과물은 사용량 벡터로 참조	5.2 계산식 없는 사실 + 계산 투영 DAG
P2	역할별 벡터를 끝까지 분리, 지급 단계에서만 합산	5.3 역할 분리 당사자 벡터
P3	유리수 정확 계산, 층별 금액 보존 반올림, 결과 digest	5.5 결정적 안분과 금액 보존
P4	범위 합성은 교집합만, 확대 금지	5.4 교집합만 허용하는 범위 대수
P5	효력 시간과 인지 시간의 분리	5.1 이중 시간축 권리 사실
P6	분쟁 그룹 보류, 재정규화 금지	5.6 분쟁 시 그룹 보류
P7	역할·자산·이벤트 종류는 프로파일 등록	5.10 확장점 레지스트리와 프로파일 바인딩
P8	허락 판정과 안분을 별 층·별 문서·별 판정으로	5.9 허락 게이트와 안분의 분리
(공통)	문서 불변, 의존 고정, 정규화 digest, 공개 등급	5.8 버전 불변 문서 봉투
(공통)	위임 연쇄의 유한성	5.7 위임 체인 제한

3. 2차 이용 권리 관리 언어 규격의 필요성

3.1 왜 "언어"인가

2장의 요구사항은 데이터베이스 스키마나 계약서 템플릿으로도 부분적으로 충족된다. 그럼에도 언어(선언적 DSL + 정규 중간 표현)를 택하는 이유는 다음 다섯 가지 성질을 **동시에** 요구하기 때문이다.

성질	의미	현행 도구가 부족한 이유
선언적	규칙을 "무엇"으로 쓰고 "어떻게"는 평가기가 정한다	정산 스크립트는 절차이므로 같은 의미가 여러 코드로 쓰이고, 검토자가 의미를 복원해야 한다. 계약서는 해석이 필요하다
버전 고정	`(module, id, version)`은 불변, 수정은 새 버전	데이터베이스 행은 갱신된다. 어느 시점에 어떤 규칙이 적용됐는지 복원이 어렵다. 계약 개정본 관리는 수작업이다
해시 가능	정규 직렬화 후 digest로 동일성 판정	계약서 파일의 해시는 내용 동일성이 아니라 파일 동일성이다. 공백·순서가 다른 두 규칙이 같은 의미인지 판정할 수 없다

성질	의미	현행 도구가 부족한 이유
결정적	같은 중간 표현 + 같은 입력 → 같은 결과 + 같은 digest	스크립트는 실행 시각, 부동소수, 외부 조회에 의존하기 쉽다. 단체 배분은 내부 엔진에 닫혀 있다
감사 가능·이식 가능	제3자가 레퍼런스 평가기로 재계산해 digest를 대조. 프로파일 간 공통 코어	스키마는 특정 운영자 모델에 묶인다. ODRL은 판정은 이식 가능하나 금액을 다루지 않는다

[N-3.1] 준수 구현은 텍스트 문법·입력 화면·저장 테이블을 **중간 표현(IR)에서 파생**해야 한다. IR이 정본이다. 운영 저장소의 규칙성 행은 IR 컴파일 결과로만 생성되어야 하며, 게시된 IR을 직접 고쳐서는 안 된다.

3.2 법적 판단과 계산의 분리

언어는 다음을 **판정하지 않아야 한다**(MUST NOT).

- 2차적저작물이 법적으로 성립하는가, 누구에게 귀속되는가.
- 어떤 권리 주장이 진실인가. `VERIFIED`는 정해진 검토 절차 통과를 뜻하며 법원 판단이 아니다.
- 요율·비율 협상.
- 세무·원천징수.

언어는 **"승인된 사실과 정책을 입력으로, 결정적으로 계산하고 그 과정을 증명한다"** 에 그친다. 이 분리가 없으면 계산 결과가 법적 판단의 대체물로 오해된다(P8). 분리는 문서 구조(모듈), 판정 구조(허락 `Decision` vs 안분 `lines/holdings`), 오류 코드 네임스페이스(`GRANT_` vs `ALLOC_`)에서 모두 드러나야 한다.

현행 체계는 이 분리를 언어 수준에서 강제하지 않는다. 계약서는 성립·허락·대가를 한 문서에 쓰고, 정산 스크립트는 허락 여부를 주석으로만 남기며, ODRL은 허락만 다루어 대가 계산과 연결되지 않는다.

3.3 현행 수단과의 비교

열은 현행 권리 관리 수단이고, 행은 2장의 문제다. 칸은 그 수단이 문제를 **닫는가**를 적는다.

문제	계약서 템플릿	신탁단체 배분 규정	식별자·메타데이터 (DDEX/CWR)	권리표현언어 (ODRL 등)	플랫폼 정산 스크립트	본 규격
P1 연쇄 추적	건별 재서술. 정정 전파 없음	1차 작품/실연 단위	정적 크레딧 운반. 사용 관계 없음	자산 참조는 가능, 투영 DAG 없음	결과물마다 경로 복제	사실은 한 문서, 결과물은 사용량 벡터
P2 역할 분리	역할이 조항마다 다름	제도별 분리, 한 정산 라인으로는 보존 안 됨	역할 코드는 운반, 계산 키는 아님	없음	당사자 합계가 흔함	역할별 벡터·라인 필수
P3 반올림·보존	없음	고정 비율, 깊이 곱셈 잔여 규칙 비공개	없음	없음	구현마다 다름, 재현 곤란	유리수 + 계층 최대잔여 + 층별 보존
P4 범위 승계	문장, 기계 검증 없음	일률 이용 유형	거래 조건 통지는 가능, 교집합 연산 없음	충돌 전략은 있으나 meet-only 고정 없음	검색 상한과 허락이 혼선	Scope meet-only, 확대 = 오류
P5 이중 시간	효력 기간만	현재 등록 조회	개정 파일은 이력, 질의 이축 없음	기간 Constraint	실행 시각 의존	`(as_of, known_at)` 코어
P6 분쟁 보류	에스크로 조항 (비계산)	전체 정지 또는 비례 축소	없음	없음	재정규화하기 쉬움	그룹 보류 키, 재정규화 금지
P7 신규 역할	조항 추가	기존 역할에 매핑	어휘 고정	프로필로 어휘 확장은 가능, 계산과 비결합	하드코드	확장점 등록, 코어 불변식 유지
P8 허락≠안분	한 문서에 혼재	분배만	없음	허락만	한 함수에 혼재하기 쉬움	별 모듈·별 판정·별 오류 접두
제3자 재계산	불가	불가(내부 엔진)	해당 없음	판정은 가능, 금액 불가	코드·입력 비정규	공개 번들 + 레퍼런스 평가기 + digest
도입 비용	낮음	(이미 존재)	(이미 존재)	중간	낮음	높음(규격·평가기·적합성)

본 규격은 도입 비용이 가장 높다. 그 비용이 정당화되는 조건은 **다수 당사자가 같은 계산을 독립적으로 검증해야 하고, 결과물 수가 많고, 권리 사실이 사후 변경되는** 환경이다. n차 음악 창작은 이 세 조건을 동시에 갖는다.

[N-3.2] 준수 구현은 위 표의 "본 규격" 열에 적은 성질을 코어에서 충족해야 한다. 프로파일은 열의 성질을 약화해서는 안 된다.

3.4 마스터와 프로파일

세 가지 언어 구조를 검토한다.

대안	장점	단점	판단
(a) 하나의 큰 DSL에 모든 창작 모드 어휘를 넣는다	단일 문법, 단일 파서	한 모드의 자산 모델이 다른 모드에 강제된다. 어휘 추가마다 코어 버전이 오른다	기각
(b) 창작 모드마다 독립 DSL을 만든다	각 모드에 최적	시간축·digest·반올림·분쟁 처리가 모드마다 달라져, 1장의 빈 층이 제품 수만큼 재발한다	기각
(c) 마스터 코어 + 확장점 + 프로파일	코어 불변식은 한 번 검증, 어휘는 모드별 등록. 프로파일 간 결과 digest 비교 가능	확장점 경계 설계가 어렵다. 프로파일이 코어를 우회할 유혹	**채택**

[N-3.3] 프로파일은 코어 불변식을 비활성화하거나 완화해서는 안 된다. 강화·추가만 허용한다.

[N-3.4] 검증되지 않은 일반화를 코어에 넣어서는 안 된다. 문법·스키마가 고정되지 않은 항목은 "방향 확정(문법 미고정)"으로 표시한다.

코어에 둘 것(시간축, digest, 금액 보존, 분쟁 보류, 층 분리)과 프로파일에 둘 것(자산 종류, 역할 이름, 사용량 메트릭, 분할 층 구조)의 경계는 5.10절과 6.7절이 정한다.

---

4. 설계 목표 및 방향

4.1 목표

#	목표	판정 기준
G1	**결정성**	동일 IR digest + 동일 입력 digest → 동일 결과 digest. 실행 시각·난수·외부 입출력을 평가에 써서는 안 된다
G2	**금액 보존**	모든 분할 층에서 Σ 하위 = 상위. 위반 시 결과 전체 무효
G3	**감사 가능성**	제3자가 공개 번들만으로 구조·불변식·digest 체인을 검증하고, 레퍼런스 평가기로 재계산할 수 있다
G4	**층 분리**	권리 사실 / 이용허락 / 안분 정책 / 정산 원장이 별 모듈·별 판정·별 오류 네임스페이스다
G5	**확장성**	새 창작 모드는 확장점 등록만으로 추가되고 코어 버전을 올리지 않는다
G6	**닫힌 거절(fail-closed) 허락**	판정 불가는 DENY다. 안분 결과의 존재가 허락을 대체하지 않는다
G7	**프로파일 독립성**	한 프로파일의 자산·역할 어휘가 다른 프로파일에 강제되지 않는다

4.2 비목표

본 규격과 그 준수 구현은 다음을 **하지 않는다**.

1. 2차적저작물의 법적 성립·귀속 판정.
2. 권리 주장의 진실성 판정(검토 절차 통과 상태의 기록만 한다).
3. 요율·비율 협상, 계약 조건 생성.
4. 세무·원천징수·회계 계정 매핑(원장 접점에서 값을 넘기기만 한다).
5. 지급 실행(결제 사업자 호출, 계좌 이체). 계산 인터페이스는 지급을 부수효과로 수행해서는 안 된다.
6. 임베딩·음성 유사도. 본 규격의 "벡터"는 역할별 지분 벡터만을 뜻한다.
7. 분산원장 실행. digest를 외부에 고정하는 일은 선택이며 본 규격의 범위 밖이다.

4.3 설계 원칙

1. **IR이 정본이다.** 텍스트 문법·입력 화면·저장 테이블은 IR에서 파생된다.
2. **사실과 정책의 분리.** 사실 문서에는 계산식이 없다. 정책 문서는 사실을 digest로 참조한다.
3. **결정성.** 4.1 G1.
4. **좁히기만 가능.** 범위(Scope)의 합성은 교집합, 계승은 축소 전용. 확대는 컴파일 오류다.
5. **보류는 재분배하지 않는다.** 분쟁·부정 의심·미확인 몫은 보류 계정으로 가고 남은 당사자에게 재정규화하지 않는다.
6. **명시 필수, 암묵 기본값 금지.** 정산 결과에 영향을 주는 값의 생략을 허용하는 프로파일은 그 사실을 프로파일 규격에 명시해야 한다(SHOULD). 마스터는 생략 시 오류인 항목을 코어에 둘 수 있다.
7. **프로파일은 코어를 약화할 수 없다.** 코어 불변식의 비활성화·완화 금지. 추가·강화만 가능.
8. **확정된 것만 코어에 둔다.** 문법 미고정 항목은 코어 불변식으로 승격하지 않는다.

9. **오류는 코드로 말한다.** 모든 판정·검증 결과는 `{ result, reason_codes[], trace }`이고 코드는 `<LAYER>_<CODE>`다.
10. **어댑터는 변환만 한다.** DDEX·ODRL·식별자 어댑터는 규칙을 판정하지 않는다.

4.4 문서 우선순위 규칙

[N-4.1] 마스터 규격(본 문서)과 프로파일 규격이 **코어 불변식**에서 충돌하면 마스터가 우선한다.

[N-4.2] 마스터 규격과 프로파일 규격이 **프로파일 어휘·인코딩·창작 모드 결정**에서 충돌하면 프로파일이 우선한다. 마스터는 프로파일 어휘를 재정의해서는 안 된다.

[N-4.3] 프로파일 규격과 구현이 충돌하면 규격이 우선하고 구현을 고친다. 구현이 규격보다 더 엄격한 검증을 추가한 경우(알 수 없는 키 거부 등)는 충돌이 아니라 강화로 본다.

[N-4.4] 이 규칙에 따라 해소되지 않는 충돌은 새 마스터 버전 또는 새 프로파일 버전으로만 해소하며, 문서 본문의 조용한 수정으로 해소해서는 안 된다.

4.5 버전 정책

대상	표기	규칙
마스터 규격	`v0.x` 초안 → `v1.0`	`v1.0`은 서로 다른 두 프로파일이 코어를 공유함을 확인한 뒤 발행한다 (8.3절)
프로파일 규격	프로파일 이름 + 버전	프로파일은 대상 마스터 버전을 선언해야 한다
언어 버전(IR 봉투)	`lang: "MAJOR.MINOR"`	비호환 변경은 MAJOR. 준수 구현은 자신이 받는 버전 집합을 명시해야 한다
확장점 레지스트리	레지스트리 버전 문자열	값 추가·변경은 레지스트리 버전을 올려서만 한다
결과 IR 형식	`format` 태그(`projection/1`, `allocation-result/1`)	형식 변경 시 숫자를 올린다. 태그는 digest에 포함되어야 한다
문서 인스턴스	`(module, id, version)`	게시 후 불변. 수정은 새 version + `superseded_by`

[N-4.5] `version`은 양의 정수 문자열이어야 한다. 게시한 문서를 같은 `version`으로 고쳐 써서 재게시해서는 안 된다.

[N-4.6] 봉투의 언어 버전 필드 이름은 `lang`이다. 영문 가칭이 확정되면 필드 이름을 가칭에 맞출지 여부는 별도 결정이다. 필드 이름을 바꾸면 모든 IR digest가 바뀌므로 MAJOR다.

---

5. 핵심 개념 (특허적 요소)

각 개념은 **정의 → 해결하는 문제 → 기존 체계가 해결하지 못하는 이유 → 규범 문장 → 규격 성숙도** 순으로 적는다.

5.1 이중 시간축 권리 사실 (as_of / known_at)

정의. 모든 권리 사실 레코드(claim, mandate)는 두 시간축을 가진다. **효력 구간** `valid [from, to)` 은 "현실에서 언제 참인가"이고, **기록 구간** `recorded_at / superseded_at` 은 "시스템이 언제부터 언제까지 그렇게 알고 있었는가"다. 질의는 `(as_of, known_at)` 두 시점을 받는다. 레코드는 `recorded_at ≤ known_at < superseded_at` 이고 `as_of ∈ valid` 일 때 계산 대상이 된다.

해결하는 문제. P5. 늦게 등록된 소급 권리 이전이 있을 때, "당시 판단의 재현"(`known_at`을 당시로)과 "현재 지식으로 당시 이용을 재계산"(`known_at`을 지금으로)을 같은 사실 문서에서 모두 얻는다. 어느 것을 정산에 쓸지는 정책의 `bind.time`(`ISSUE_TIME | USAGE_TIME | SALE_TIME | REPORT_PERIOD`)이 정하고, 호출자가 그 규칙대로 `as_of`를 정한다.

기존 체계가 해결하지 못하는 이유. 이중 시간(bitemporal) 모델 자체는 SQL:2011의 application-time/system-time 테이블로 표준화된 개념이다(부록 C). 집중관리 등록과 CWR 개정은 보통 **현재 유효 지분**만 조회하거나, 개정 이력을 파일 단위로 남긴다. 정산 스크립트는 실행 시각에 의존한다. 어느 것도 (i) 해시 고정된 선언 문서의 각 주장에 이축을 두고, (ii) `known_at` 시점의 가시 집합을 투영 결과 digest에 포함시키며, (iii) 정적 검증(합 10000bps 등)을 "현재 지식(미대체) 기준으로 모든 유효기간 경계 시점"에서 수행하지 않는다. 단일 시간축 스냅샷은 감사 재현은 되지만 소급 정정을 같은 문서로 표현하지 못한다.

규범 문장.

- [N-5.1.1] 모든 권리 사실 레코드는 `valid`(반열린 구간)와 `recorded_at`을 가져야 하며, 대체 시 `superseded_at`을 기록하고 원 레코드를 삭제해서는 안 된다.
- [N-5.1.2] 투영·평가는 `(as_of, known_at)`을 입력으로 받아야 하며, 두 값이 같은 입력은 영구히 같은 결과를 내야 한다.
- [N-5.1.3] 투영 결과 IR은 `as_of`, `known_at`, 사실 문서 digest를 포함해야 하고 이들은 결과 digest에 포함되어야 한다.

- [N-5.1.4] 상태 전이는 `ASSERTED → VERIFIED | DISPUTED`, `* → SUPERSEDED`이며 투영 대상은 `VERIFIED`(및 보류 처리를 위한 `DISPUTED`)만이다.

규격 성숙도. 확정.

5.2 계산식 없는 사실 주장 + 계산 투영 DAG

정의. 사실 문서(facts)는 엔티티(당사자·자산)와 **주장(claim)·위임(mandate)** 만을 담고, 계산식·비율 정책·조건문을 담지 않는다. 계산은 별도 단계인 **투영(projection)** 이 한다. 투영은 자산에서 당사자로 가는 역할별 경로를 따라 `w_role[p] = Σ_s u[s] · Π_{e∈path} share(e)/10000` 을 정확한 유리수로 계산하고, 계산에 쓰이는 경로 그래프는 DAG여야 한다.

해결하는 문제. P1. 결과물 수가 많아도 사실은 한 문서에 있고, 결과물은 **희소 사용량 벡터 `u`** 로만 참조한다. 사실이 정정되면 새 버전의 사실 문서로 재투영하면 되고, 결과물마다 복제된 경로를 고칠 필요가 없다. 사실에 계산식이 없으므로 사실 문서의 공개 등급을 독립적으로 관리할 수 있다(5.8절).

기존 체계가 해결하지 못하는 이유. 권리 그래프와 크레딧 목록은 새롭지 않다. DDEX RIN은 세션 기여자를 운반하고, CWR은 작품 지분을 운반한다. 차이는 (i) **전체 지식 그래프가 아닌 계산용 투영만 DAG를 요구**하고, (ii) 투영 결과가 `paths[]`(자산·역할·당사자·가중치·claim id 목록)까지 담아 **어느 주장이 그 가중치를 만들었는지** 추적 가능하며, (iii) 사용량 `u`를 정규화한 `u_digest`를 별도로 기록해 사용량 조작을 탐지할 수 있게 한 점이다. 식별자는 고정 작품·녹음만 가리키므로, 사용 관계를 결과물마다 복제하지 않는 정본이 없다.

규범 문장.

- [N-5.2.1] 사실 문서에는 계산식·비율 정책·조건 분기를 두어서는 안 된다. 사실 문서의 수치는 지분(`share`, bps 정수)과 시간만이다.
- [N-5.2.2] 지분을 갖는 주장은 `(source, edge, [role 세부])` 그룹으로 묶이며, 동일 시점에 유효한 그룹의 share 합은 정확히 10000bps여야 한다(`FACTS_SUM_MISMATCH`). 구조 관계 엣지는 지분을 갖지 않는다.
- [N-5.2.3] 계산용 투영 그래프는 DAG여야 한다(`FACTS_CYCLE`).
- [N-5.2.4] 투영은 `(facts@digest, as_of, known_at, u)`의 순수 함수여야 하며 결과 IR은 `vectors`, `paths`, `facts_digest`, `u_digest`, `digest`를 포함해야 한다.
- [N-5.2.5] 참조되는 당사자는 모두 레지스트리에 존재해야 한다(`FACTS_UNKNOWN_PARTY`).

규격 성숙도. 확정.

5.3 역할 분리 당사자 벡터 (PartyVector per role)

정의. 기여 벡터는 **청구권자 역할(claimant role)별로 독립된** `PartyVector<Role> = Map<PartyRef, Ratio>` 이며 각 벡터의 합은 1이다. 역할 간 사전 합산을 금지한다. 안분 정책의 각 버킷은 하나의 역할을 다루고, 결과 라인은 `(party, role, bucket)`을 키에 포함한다. 지급 단계(원장)에서만 당사자별로 합산한다.

해결하는 문제. P2. 같은 사람이 작곡가·실연자로 두 역할에서 받는 금액이 각각 어느 권리의 대가인지 끝까지 남는다. 단체 보고(작품 vs 실연), 세무 구분, 명세서 설명이 가능하다. 위임 수령인의 역할도 규칙으로 정해진다: 집중관리단체 종류면 `CMO`, 에이전시 종류면 `AGENCY`, 그 외 kind는 원래 역할을 유지한다(예: 작가 몫을 받는 출판사는 `WORK_AUTHOR`).

기존 체계가 해결하지 못하는 이유. 통상의 정산은 당사자별 합계 금액을 산출한다. 단체는 제도별로 분배하나 한 결과물의 한 실행에서 역할 키를 라인 유일키에 넣지 않는다. DDEX 역할 코드는 보고용이지 계산 타입 불변식이 아니다. 본 규격은 (i) 역할 분리를 **타입 시스템 불변식**으로 올리고, (ii) 버킷 역할과 벡터 잎 역할의 일치를 정적 검증하며, (iii) 실연 라인의 역할을 정책이 아니라 **투영에서 도출**한다.

규범 문장.

- [N-5.3.1] `PartyVector`는 역할별로 독립이어야 하며, 역할이 다른 벡터를 합산하는 연산은 타입 오류다.
- [N-5.3.2] 안분 결과의 라인 키는 최소 `(event, party, role, bucket, line_seq)`를 포함해야 한다.
- [N-5.3.3] 버킷의 역할과 그 벡터 식이 도달하는 역할은 일치해야 한다(`CORE_TYPE_ERROR`).
- [N-5.3.4] 프로파일은 역할 어휘를 확장점에 등록하며, 마스터는 역할의 개수·이름을 정하지 않는다(6.7절).

규격 성숙도. 확정.

5.4 교집합만 허용하는 범위 대수 (Scope meet-only)

정의. `Scope = { actions, territories, media, period, audience }` 는 격자(lattice)를 이루고, 합성 연산은 **meet(교집합, ⊓)만** 허용한다. 이용허락 판정에서 유효 범위는 `grant ⊓ 하위조항 ⊓ 운영범위` 이고, 부정(deny)이 우선하며, 의무는 합집합, 범위 확대는 `GRANT_SCOPE_WIDENING` 오류다. 정책 템플릿 계승에서도 확대는 `ALLOC_WIDENING`이다.

해결하는 문제. P4. n차 결과물의 허락 범위는 재료들의 범위 교집합으로 **기계적으로** 좁아진다. 2.4절 예에서 `{국내, 해외} ⊓ {국내} ⊓ {국내, 숏폼} = {국내 앱}`. 상위 단계로 내려갈수록 좁아지기만 하므로, 하위 문서가 상위 허락을 넘는 일이 문법 수준에서 불가능하다.

기존 체계가 해결하지 못하는 이유. ODRL은 Permission과 Prohibition을 나열하고 충돌 전략(`conflict`)을 정하지만, 정책 간 범위 합성을 격자 연산으로 고정하지 않는다. 약관은 일률 범위이고, 검색용 후보 상한(합집합)과 실제 허락을 같은 값으로 쓰기 쉽다. 계약서는 축소 전용을 문장으로만 쓴다. 본 규격은 join(합집합)을 표시용 파생값으로만 허용하며 허락 판정 IR에 나타날 수 없게 한다.

규범 문장.

- [N-5.4.1] `Scope` 합성은 meet만 허용한다. join을 허락 판정의 입력으로 쓰는 문서는 컴파일 오류다.
- [N-5.4.2] 하위 문서의 범위는 상위 문서 범위의 부분집합이어야 한다. 위반은 `GRANT_SCOPE_WIDENING` 또는 프로파일이 정한 상응 코드다.
- [N-5.4.3] 하위 표지(스탬프·고지 등)가 행위 코드를 쓰는 프로파일에서, 그 표지는 상위 허락 행위의 부분집합이어야 한다.
- [N-5.4.4] 부정 우선(deny-overrides), 의무 합집합.

규격 성숙도. 방향 확정(문법 미고정). 의미론은 확정, `Scope` 타입과 meet 연산의 IR·구현은 이후 버전.

5.5 결정적 안분과 금액 보존 (계층적 최대잔여)

정의. 평가기는 순수 함수 `evaluate(policy_IR, vectors_IR, snapshot, revenue_events[]) → { lines[], holdings[], trace, result_digest }` 다. 중간 계산은 큰 정수 유리수(`Ratio`)로 정확히 수행하고, 금액은 통화 최소 단위 정수(`Money<C>`)다. 각 분할 층에서 상위 정수 금액 `R`을 하위 비율 `r_i`로 나눌 때 `a_i = floor(R·r_i)`, 잔여 `Δ = R − Σa_i`를 소수부 내림차순·동률 시 키 오름차순으로 1씩 배분하고, 층마다 `Σa_i = R`을 검증한다. 방법의 레지스트리 이름은 `HIERARCHICAL_LRM_V1`이다.

해결하는 문제. P3. 유리수 분모가 커져도 정확하고, 어느 층에서도 1 최소 단위의 누출·중복이 없으며, 같은 입력은 같은 `result_digest`를 낸다. 공개 검증용 구현과 운영 구현을 분리하지 않으므로, 검증 결과가 실제 지급과 어긋날 여지가 없다.

기존 체계가 해결하지 못하는 이유. 최대 소수잔여법(largest remainder, Hamilton 방법)은 1792년에 제안된 의석 배분 방법으로 잘 알려져 있다(부록 C). 그 자체는 선행 기술이다. 단체 요율과 정산 스크립트는 한 층의 정수 분배만 하거나, 잔여를 최대 지분자에게 몰아주거나, 부동소수로 처리한다. 차이는 (i) **계층형** 적용(각 층에서 상위 확정 정수 금액을 다시 분배하고 층마다 보존식을 검증), (ii) 동률 규칙을 **안정 키 순서**로 고정해 결정성을 보장, (iii) 분쟁 보류 키 `@hold`를 배분 대상에 포함시켜 벡터 합 1을 유지하면서 보류 금액도 같은 규칙으로 정수화, (iv) 결과 전체의 `result_digest`와 입력 digest들(`policy_digest`, `projection_digest`, `snapshot_digest`, `events_digest`)을 함께 기록한 점이다.

규범 문장.

- [N-5.5.1] 평가는 순수 함수여야 한다. 실행 시각, 난수, 외부 입출력, 부동소수를 사용해서는 안 된다.
- [N-5.5.2] 비율은 정수 bps, 중간 계산은 기약 유리수, 금액은 통화 최소 단위 정수여야 한다. IR에서 JSON number를 사용해서는 안 되며 모든 수치는 문자열이다(`CORE_NUMBER_IN_IR`).
- [N-5.5.3] 모든 분할 층에서 `Σ 하위 = 상위`가 성립해야 하고, 위반 시 결과 전체가 무효다(`ALLOC_ROUNDING_INVARIANT`).
- [N-5.5.4] 반올림 방법은 정책 IR에 이름으로 명시되어야 하며(`rounding.method`), 마스터는 방법 레지스트리에 `HIERARCHICAL_LRM_V1`을 등록한다. 새 방법은 새 이름으로만 추가하고 기존 결과를 바꾸지 않는다.
- [N-5.5.5] 결과 IR은 입력 digest들과 `result_digest`, 형식 태그를 포함해야 한다.
- [N-5.5.6] 통화가 다른 값끼리의 연산은 타입 오류다. 이벤트의 통화·유형은 정책 scope 안이어야 한다(`ALLOC_SCOPE_MISMATCH`).

규격 성숙도. 확정.

5.6 분쟁 시 그룹 보류·재정규화 금지

정의. 지분 그룹 안에 `DISPUTED` 주장이 하나라도 있으면 그 **그룹 전체**를 보류한다. 보류분은 벡터에서 `@hold` 키로 남아 벡터 합 1을 유지하며, 남은 당사자에게 재정규화하지 않는다. 정산 시 `@hold` 몫은 `holding(reason = RIGHTS_DISPUTE)`로 간다. 분쟁 그룹은 10000bps 합 검사에서 제외한다. 분쟁 중인 주장·위임이 권리 경로에 있으면 새 결과물의 활성화를 막는다(`FACTS_DISPUTE_OPEN`).

해결하는 문제. P6. 분쟁 당사자의 몫이 타인에게 선지급되지 않고, 해소 후 보류분만 처리하면 되므로 역분개 범위가 최소화된다. 부정 의심 이벤트(`integrity = FLAGGED`)는 1차 분할 층은 기록하고 권리자 풀 전액을 `FRAUD_REVIEW`로 보류한다.

기존 체계가 해결하지 못하는 이유. 단체 운영은 곡 전체 지급 정지 또는 초과 청구의 비례 축소에 가깝다. 정산 스크립트는 "제외 후 재정규화"를 택하기 쉽다. 계약 에스크로는 계산 벡터의 원소가 아니다. 본 규격은 그룹 단위 보류를 **벡터 연산 규칙**으로 고정한다: `@hold`는 상한·최저보장·재분배 대상이 아니다.

규범 문장.

- [N-5.6.1] 분쟁 그룹의 몫은 보류 키로 벡터에 남아야 하며 다른 당사자에게 재분배해서는 안 된다.
- [N-5.6.2] 보류 키는 청구권자가 아니며, 상한·최저보장·재분배 연산의 대상이 아니다.
- [N-5.6.3] 보류는 사유 코드(`holding_reason`)와 함께 결과 IR에 기록되어야 한다.
- [N-5.6.4] 분쟁 중인 권리 경로를 쓰는 새 결과물의 활성화는 거부되어야 하며 거부 사유는 분쟁 레코드 id 목록을 포함해야 한다.
- [N-5.6.5] 보류 해제·정정은 원 결과를 수정하지 않고 새 실행(run)으로 처리한다(원장 접점, 6.6절).

규격 성숙도. 확정.

5.7 위임 체인 제한 (mandate ≤ 2홉)

정의. `mandate(from, to, right, share, valid, recorded_at)`는 위임인이 받을 몫 중 수임인이 수령하는 비율을 권리 종류별로 표현한다. 위임인별·권리별 위임 합은 10000bps 이하다. 위임 연쇄는 최대 2홉이며 3홉 이상은 `FACTS_MANDATE_DEPTH`로 거부한다. 실제 3홉 이상 위임은 등록 단계에서 단일 홉으로 평탄화한다.

해결하는 문제. 1.2절의 "권리자 ≠ 수령자" 관계를 사실로 기록하되, 위임 순환·무한 연쇄로 투영이 발산하는 것을 막는다. 순환은 `FACTS_CYCLE`. 위임의 `share`도 이중 시간축·분쟁 상태를 가지므로 위임 자체가 다투어질 때 그 몫이 `@hold`로 간다.

기존 체계가 해결하지 못하는 이유. 단체 등록의 "신탁 여부" 불리언은 권리 종류·지역·기간·지분별 차이를 잃는다. CWR은 출판사·작가를 나열하나 위임을 계산 그래프의 엣지로 두지 않는다. 홉 수 상한을 **언어 불변식**으로 두는 것은 실제 위임 구조의 복잡성을 등록 단계로 밀어내는 설계 결정이다.

규범 문장.

- [N-5.7.1] 위임은 권리 종류별로 선언되어야 하며, 위임인별·권리별 합은 10000bps를 넘어서는 안 된다(`FACTS_SUM_MISMATCH`).
- [N-5.7.2] 위임 연쇄 깊이는 마스터가 정한 상한(현재 2)을 넘어서는 안 된다(`FACTS_MANDATE_DEPTH`). 프로파일은 상한을 낮출 수 있으나 높일 수 없다.
- [N-5.7.3] 위임 순환은 오류다(`FACTS_CYCLE`).
- [N-5.7.4] 위임 수령인의 라인 역할은 수령인 kind가 집중관리·에이전시이면 그 kind, 그 외에는 위임인의 역할이다. kind 어휘는 프로파일 등록값이다.

규격 성숙도. 확정. 상한 2를 마스터 상수로 고정할지, 프로파일 파라미터(상한 ≤ 마스터 최대)로 둘지는 8.4절.

5.8 버전 불변 문서 봉투 + 의존 digest 고정 + JCS + 공개 등급/커밋먼트

정의. 모든 문서는 봉투 `{ lang, module, id, version, depends_on[{ref, digest}], visibility_default, body }`를 가진다. 게시된 `(module, id, version)`은 불변이다. 의존은 `rl:<module>/<id>@<version>` 참조와 **그 대상의 digest**로 고정된다. 직렬화는 RFC 8785 JCS이고 `digest = "sha256:" + hex(sha256(UTF-8(JCS(IR))))`다. 모듈 의존은 낮은 순위(rank)로만 허용된다(`MODULE_RANK`: core 0 < facts 1 < constraint·grant 2 < approval 3 < allocation 4; 역방향·동일 rank는 `CORE_DEP_DIRECTION`). 모든 노드는 `visibility ∈ {PUBLIC, PARTIES, PRIVATE}`를 가질 수 있고, 공개본에서 비공개 값은 `{ commit: sha256(salt ‖ JCS(value)) }`로 치환된다. salt는 값마다 32바이트 난수이며 당사자에게만 전달된다.

해결하는 문제. 1.6.3의 불투명성. 제3자는 구조·digest 체인·서명·공개된 합 불변식을 검증하고, 당사자는 salt로 자기 값만 열어 재계산까지 한다. 참조 대상의 새 버전이 나와도 기존 문서의 의미가 변하지 않는다. 텍스트의 공백·주석·선언 순서는 digest에 영향을 주지 않는다.

기존 체계가 해결하지 못하는 이유. 콘텐츠 해시·정규화 직렬화 각각은 선행 기술이다(RFC 8785, 부록 C). 계약서 PDF 해시는 파일 동일성이고, 단체 명세서에는 재계산 입력이 없으며, ODRL 정책 `uid`는 의존 digest를 고정하지 않는다. 차이는 (i) **모듈 rank 방향 제약**으로 의존 그래프를 자동으로 비순환으로 만들고, (ii) 공개 등급 치환 후의 **공개 digest와 원문 digest를 함께** 목록에 기록하며, (iii) 사실 문서의 `share`·`evidence`는 문서 기본값과 무관하게 항목별 `PUBLIC` 명시가 없으면 커밋먼트로 치환하는 **안전 기본값**, (iv) 서명 메시지에 `signer, role, over, alg`를 함께 묶어 역할·서명자 바꿔치기를 막는 규칙, (v) 결과 IR의 형식 태그를 digest에 포함한 점이다.

규범 문장.

- [N-5.8.1] 봉투 필드는 위 7개이며 알 수 없는 키는 거부한다.
- [N-5.8.2] `version`은 양의 정수 문자열이고 게시 후 불변이다. 수정은 새 version과 `superseded_by`로만 한다.
- [N-5.8.3] `depends_on`의 각 항목은 `ref`와 대상 문서의 `digest`를 가져야 하며, 불일치는 `CORE_DIGEST_MISMATCH`, 대상 없음은 `CORE_UNKNOWN_REF`, 허용되지 않는 방향은 `CORE_DEP_DIRECTION`이다.
- [N-5.8.4] 직렬화는 JCS, 수치는 문자열, 순서가 의미 없는 컬렉션은 키 정렬(사실 문서의 엔티티 배열은 id 오름차순).
- [N-5.8.5] 공개본 생성 시 `PUBLIC`이 아닌 값은 커밋먼트로 치환하고, 공개 digest와 원문 digest를 함께 기록한다.
- [N-5.8.6] 서명은 `JCS({ctx, signer, role, over, alg})`의 UTF-8 바이트에 대해 이루어져야 한다.

규격 성숙도. 확정.

5.9 허락 게이트와 안분의 분리 (닫힌 거절)

정의. 이용허락 판정은 `Requirement(asset, right_type, scope)` 집합에 대해 요구사항 간 AND, 동일 요구를 충족하는 대체 허락 간 OR로 평가하고, `PERMIT | PERMIT_WITH_OBLIGATION | DENY | REVIEW_REQUIRED` + `reason_codes` + `trace`를 낸다. **판정 불가는 `DENY`** 다. 안분은 별 모듈(`allocation`)이며 허락을 판정하지 않는다. 안분 정책이 grant를 참조하는 문법은 grant 스키마 확정 시 열린다.

해결하는 문제. P8. 허락 없는 이용의 매출도 계산할 수 있지만 그 결과는 사고 처리 근거일 뿐이다. 두 판정이 별 문서·별 오류 네임스페이스·별 결과 형태이므로 "계산됨 = 허락됨" 추론이 구조적으로 성립하지 않는다. `REVIEW_REQUIRED`는 외부 이용 실행 관점에서 불허다.

기존 체계가 해결하지 못하는 이유. 계약서는 허락과 대가를 한 문서에 쓴다. 정산 스크립트는 금액만 내고 허락을 주석으로 남긴다. ODRL은 활성 Permission이 없으면 거부하는 닫힌 거절과 가깝지만 금액 계산이 없다. 본 규격은 분리를 **언어 층 분리**로 올리고, 금액 계산은 ODRL 밖에 둔다.

규범 문장.

- [N-5.9.1] 허락 판정과 안분 계산은 별 모듈이어야 하며, 안분 결과의 존재·금액은 어떤 허락 판정의 입력이 되어서는 안 된다.
- [N-5.9.2] 허락 판정 불가(요구사항 미커버, 범위 확대, 증빙 없음)는 `DENY`여야 한다.
- [N-5.9.3] 허락 판정 결과는 `{ result, reason_codes[], trace }`이며 `trace`는 선택된 허락 경로를 포함해야 한다.
- [N-5.9.4] 횟수 제한 허락은 평가 → 원자적 예약 → 실행 → 소비/해제 순서를 따라야 한다. 문법은 본 버전에서 미고정.
- [N-5.9.5] 철회는 `revocation_epoch`로 전파되어야 한다. 문법은 본 버전에서 미고정.

규격 성숙도. 방향 확정(문법 미고정). 층 분리·오류 접두·안분이 facts에만 의존하는 코어 측면은 확정.

5.10 확장점 레지스트리와 프로파일 바인딩

정의. 마스터 코어는 **어휘 확장점**만 정하고 값을 정하지 않는다: 자산 종류, 엣지 종류, 청구권자 역할, 사용량 메트릭, 벡터 함수, 이벤트 종류, 풀 모드, 반올림 방법, 보류 사유. 프로파일은 각 확장점에 값을 **등록**하고, 코어 검증기는 "등록된 값만 사용"을 강제한다(`CORE_ENUM_UNKNOWN`).

해결하는 문제. P7. 팬 기여자·AI 제공자·플랫폼 풀처럼 기존 표준에 없는 역할을 코어에 넣지 않고도 등록할 수 있고, AI 믹스 프로파일은 특정 자산 모델 없이 자기 자산 종류를 등록한다. 코어 불변식(합 10000bps, DAG, meet-only, 계층 최대잔여 보존)은 어휘와 무관하게 작동한다.

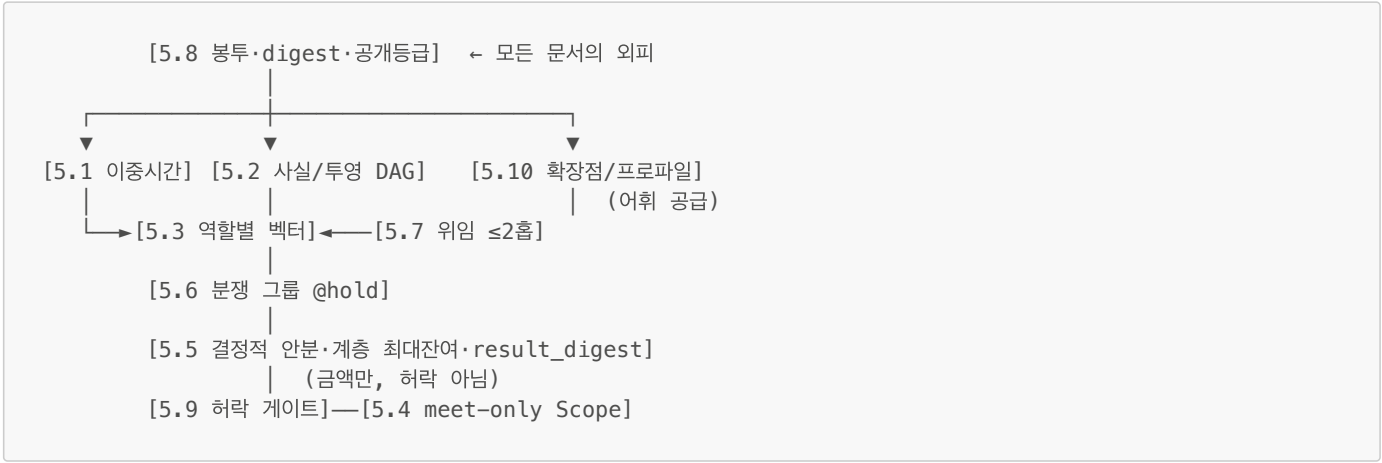
기존 체계가 해결하지 못하는 이유. ODRL의 Profile 메커니즘(`odrl:profile`)과 MPEG-21 RDD의 등록기관 방식도 어휘 확장을 둔다. DDEX 역할 코드는 보고 어휘다. 차이는 (i) 확장점이 **계산 의미론과 결합**되어 있다는 점(벡터 함수는 `PartyVector`를 반환해야 하고 합 1을 유지해야 한다 — `ALLOC_WEIGHT_MISMATCH`), (ii) 프로파일이 코어 불변식을 약화할 수 없다는 규칙(4.3 원칙 7), (iii) 프로파일 등록값 변경이 레지스트리 버전을 올려야 하고 그 버전이 IR 검증에 쓰인다는 점이다.

규범 문장.

- [N-5.10.1] 마스터 코어는 확장점의 **이름·타입·불변식**을 정하고 값을 정하지 않는다.
- [N-5.10.2] 프로파일은 각 확장점에 값을 등록하며, 등록되지 않은 값은 `CORE_ENUM_UNKNOWN`이다.
- [N-5.10.3] 벡터 함수 확장은 `PartyVector` 반환과 합 1 유지를 보장해야 하며 `@hold`를 재분배해서는 안 된다.
- [N-5.10.4] 프로파일은 코어 불변식을 비활성화·완화할 수 없다. 강화·추가만 가능하다.
- [N-5.10.5] 프로파일 등록값의 변경은 레지스트리 버전을 올려야 한다.

규격 성숙도. 레지스트리 강제와 벡터 함수 합 검사는 확정. 코어/프로파일 레지스트리의 물리적 분리는 방향 확정.

5.11 개념 간 결합



5.1·5.2·5.3·5.6·5.7은 facts 층, 5.5는 allocation 층, 5.4·5.9는 grant 층, 5.8·5.10은 코어에 속한다.

5.12 특허 관점 메모

이 절은 설계 지침이며 법률 자문이 아니다. 청구항 작성·선행기술 조사·자유실시 판단은 변리사 검토가 필요하다.

단독으로는 선행 기술일 가능성이 높은 요소.

- 이중 시간축(bitemporal) 모델 — SQL:2011.
- 최대 소수잔여법 반올림 — Hamilton/Hare-Niemeyer.
- 콘텐츠 해시·정규화 직렬화·해시 체인 — RFC 8785, 일반적 머클 구조.
- 허락·금지·의무 정책 언어 — ODRL 2.2, MPEG-21 REL.
- 권리 그래프 표현 — 그래프를 쓴다 또는 규칙 언어를 쓴다만으로 신규성을 주장하지 않는다.
- 식별자·메타데이터 표준 — ISWC, ISRC, CWR, DDEX.

결합으로서 방어력이 상대적으로 높다고 보는 후보(시스템/방법 청구 관점).

1. **n차 파생 결과물에 대한 이중 시간축 권리 사실 + 결정적 계층 안분 + 결과 digest 체인.** 재료 사실 문서(`as_of`/`known_at`)의 digest를 고정 참조하는 정책 문서로, 사용량 벡터를 입력받아 역할별 벡터를 투영하고, 계층형 최대 소수잔여 반올림으로 금액을 보존하며, 입력 digest들과 결과 digest를 함께 기록하여 제3자가 재계산으로 검증할 수 있는 방법. 개별 요소는 알려져 있으나 **파생 깊이를 가로지르는 결합**과 **재계산 검증 가능성**이 결합 효과다.
2. **분쟁 그룹 보류 키를 벡터 원소로 유지하는 안분.** `@hold`가 합 1을 유지하면서 상한·최저보장·재분배에서 제외되고, 정수화 규칙을 동일하게 통과하는 구조. 재정규화 금지가 역분개 범위를 최소화한다는 효과 주장이 가능하다.

3. **범위 격자 meet-only 합성으로 파생 깊이별 허락 범위를 자동 도출하고, 안분 결과와 격리하는 닫힌 거절 게이트.** 문법 미고정이므로 실시 가능성 뒷받침은 이후 버전.
4. **공개 등급별 커밋먼트 치환 + 공개 digest/원문 digest 동시 기록 + 당사자 salt 부분 공개로, 제3자 검증과 당사자 재계산을 분리하는 공개 방식.**

주의.

- 소프트웨어 발명의 성립성 요건(하드웨어 결합·기술적 효과)은 관할별로 다르다.
- 본 문서의 규범 문장은 청구항이 아니다. 청구항 초안은 별도 문서로 작성한다.

---

6. DSL 구조 및 설계

6.1 계층 아키텍처

```
flowchart TB
  subgraph MASTER["마스터 코어"]
    ENV["문서 봉투 · 버전 불변 · depends_on digest"]
    TYPES["타입: Bps · Ratio · Money · Interval · Ref · PartyVector · Scope"]
    DIG["JCS 정규화 · digest · 공개 등급 · 커밋먼트 · 서명"]
    ERR["오류 모델 · 코드 네임스페이스"]
    REG["확장점 레지스트리 자리만"]
  end

  subgraph LAYERS["층"]
    F["facts: 사실 주장 · 이중 시간축 · 투영 DAG · 위임"]
    G["grant: 요구사항 AND/OR · meet-only Scope · 닫힌 거절"]
    A["allocation: 평가 계약 · 벡터 함수 · 계층 최대잔여 · result_digest"]
    L["ledger 접점: result_digest 인계 · 보류 사유"]
  end

  subgraph PROFILES["프로파일 예시"]
    P1["창작 모드 A — 자산·역할·메트릭 등록"]
    P2["창작 모드 B — 다른 자산 모델, 같은 코어"]
  end

  MASTER --> LAYERS
  F --> G
  F --> A
  G -. "grant 스키마 확정 후 allocation이 참조할 수 있음" .-> A
  A --> L
  REG --> P1
  REG --> P2
  P1 --> F
  P1 --> A
  P2 --> F
  P2 --> A
```

의존 방향은 `core ← facts ← {constraint, grant} ← approval ← allocation`이며 코드에서는 `MODULE_RANK`로 강제되어야 한다. `constraint`·`approval`은 프로파일이 rank 2·3에 둘 수 있는 전용 모듈이며 마스터 필수 층이 아니다.

6.2 마스터 코어

6.2.1 문서 봉투

필드	타입	규칙
`lang`	문자열 `"MAJOR.MINOR"`	언어 버전. 본 초안과 호환되는 구현은 `"1.0"`을 받아야 한다
`module`	Enum `module`	`core | facts | constraint | grant | approval | allocation`. 프로파일이 모듈을 추가할 때는 rank를 함께 등록해야 한다
`id`	`^[A-Za-z0-9._-]+$`	프로파일 내 유일. `__proto__` 금지
`version`	양의 정수 **문자열**	게시 후 불변

필드	타입	규칙
`depends_on[]`	`{ ref: "rl:<module>/<id>@<version>", digest }`	대상 digest 고정, 낮은 rank로만
`visibility_default`	Enum `visibility`	노드 `visibility` 생략 시 기본값. 단, 사실 문서의 `share`·`evidence`는 항목별 `PUBLIC` 명시가 없으면 `PARTIES`
`body`	모듈별 객체	모듈 스키마

6.2.2 타입 시스템

타입	정의	불변식
`Bps`	정수 0~10000	저장 비율은 항상 정수 bps. `%` 리터럴은 소수 둘째 자리까지 (문법 설탕)
`Ratio`	기약 유리수 `"n/d"`	중간 계산 전용. 부동소수 금지. 정수도 `"1/1"`
`Money<C>`	큰 정수, 통화 최소 단위	통화 혼합 연산은 타입 오류
`DurationMs`	0 이상 정수	
`Instant`	UTC ISO 8601, ms 정밀도	
`Interval`	`[from, to)`, `to` 생략 = 무기한	`from < to`
`Ref<T>`	`rl:<module>/<id>@<version>`	버전 없는 참조 금지
`PartyVector<Role>`	`Map<PartyRef, Ratio>`	합 = 1, 역할 간 사전 합산 금지, `@hold` 키 허용
`Scope`	`{ actions, territories, media, period, audience }`	meet만 허용 (문법 미고정, 5.4)
`Enum<Name>`	레지스트리 등록 값	레지스트리 한 곳에서만 정의

6.2.3 digest 규칙

- JCS(RFC 8785) → UTF-8 → SHA-256 → `"sha256:" + hex`.
- 수치는 문자열. JSON 중첩 깊이 256 제한, 모든 문자열은 올바른 UTF-16(고립 서로게이트 거부).
- 순서가 의미 없는 컬렉션은 정렬. 사실 문서의 엔티티 배열은 id 오름차순(코드 유닛 비교).
- 결과 IR은 `format` 태그를 digest에 포함한다.

6.2.4 공개 등급

등급	뜻	공개본 처리
`PUBLIC`	누구나 원문	원문 유지
`PARTIES`	당사자만 원문	커밋먼트로 치환, salt는 당사자에게
`PRIVATE`	발행자만	커밋먼트로 치환. 당사자 레코드는 id·kind만 공개

모듈별 기본(권고): 사실의 엔티티 존재·식별자 = `PUBLIC`, share·증빙 = `PARTIES`, 안분 비율·결과 금액 = `PARTIES`.

6.2.5 오류 코드 규약

`<LAYER>_<CODE>`. 마스터가 예약하는 접두: `CORE_`, `FACTS_`, `GRANT_`, `ALLOC_`, `PARSE_`. 프로파일 모듈은 자기 접두를 등록한다.

코드	의미
`CORE_TYPE_ERROR`	타입·형식 위반(알 수 없는 키 포함)
`CORE_NUMBER_IN_IR`	IR에 JSON number
`CORE_ENUM_UNKNOWN`	레지스트리에 없는 값
`CORE_UNKNOWN_REF`	참조 대상 없음

코드	의미
CORE_DEP_DIRECTION	허용되지 않는 의존 방향(순환 포함)
CORE_DIGEST_MISMATCH	digest 불일치
CORE_SIGNATURE_INVALID	서명 실패
CORE_SOD_VIOLATION	직무 분리 위반

6.3 facts 층 — 최소 모델과 불변식

마스터가 정하는 facts의 **추상 모델**은 다음이다. 구체 인코딩은 프로파일이 정한다.

구성요소	마스터 정의
당사자 party	{ id, kind ∈ 확장점 party_kind, ids?, visibility? }
자산 asset	{ id, kind ∈ 확장점 asset_kind, of?(상위 자산), ids? }
주장 claim	{ id, source, edge ∈ 확장점 edge_kind, target, share: Bps, valid, recorded_at, superseded_at?, status, evidence?, visibility?, 세부 역할? }
위임 mandate	{ id, from, to, right ∈ 확장점 mandate_right, share: Bps, valid, recorded_at, superseded_at?, status, visibility? }
사용량 u	Map<asset_id, DurationMs 또는 메트릭 값> (문서 밖 입력)

마스터 불변식.

- [N-6.3.1] 지분 그룹 합 10000bps(동일 시점, 미대체 레코드 기준, 모든 유효기간 경계에서 검사). 분쟁 그룹은 검사 제외·전체 보류.
- [N-6.3.2] 계산 투영 DAG.
- [N-6.3.3] 참조 무결성(당사자·자산·엣지 종류와 source/target 종류의 일치 — FACTS_EDGE_KIND).
- [N-6.3.4] 식별자 중복 금지(FACTS_DUPLICATE_ID).
- [N-6.3.5] 위임 ≤ 2홉, 순환 금지, 위임인별·권리별 합 ≤ 10000bps.
- [N-6.3.6] 투영은 (facts@digest, as_of, known_at, u)의 순수 함수. 사용량이 0인 자산은 u_digest에서 제외. 경로가 없는 역할은 vectors에 키가 없다.
- [N-6.3.7] 커버리지: 사실에서 도출된 청구권자 집합과 결과물이 선언한 청구권자 목록의 차이는 FACTS_UNCOVERED_CLAIMANT(선언했으나 도출 안 됨) / FACTS_CLAIMANT_DRIFT(도출됐으나 선언 안 됨)로 보고한다.

프로파일이 정하는 것. 자산 종류와 구조 관계, 엣지 종류와 허용 source→target 쌍, 역할 경로 정의(어느 엣지 연쇄가 어느 역할을 만드는가), 사용량 메트릭의 정의와 정규화.

6.4 grant 층 — 책임과 미결

마스터가 정하는 책임.

- 입력: 결과물의 요구사항 집합 Requirement(asset, right_type, scope), 허락 문서들(각각 Scope와 의무), 부정·철회 상태.
- 판정: 요구사항 간 AND, 대체 허락 간 OR. 범위 합성 meet-only. 부정 우선. 의무 합집합.
- 출력: { result ∈ decision, reason_codes[], trace(선택 경로) }. 판정 불가 = DENY.
- 의존: facts에 의존(자산·당사자 참조). allocation이 grant에 의존할 수 있다(스키마 확정 후).

본 버전에서 문법 미고정. 허락 문서의 텍스트 문법과 IR 스키마, 예약·소비·철회 문법, revocation_epoch 전파 규칙의 IR 표현, 공동 권리의 지분 커버리지 정량 규칙, ODRL 출력 매핑표.

미결. 동의(CONSENT)의 범위(Scope)는 grant 층 사실로 두고, 승인 절차(서명·직무 분리)는 프로파일 모듈에 두는 것을 권고한다.

6.5 allocation 층 — 평가 계약

평가 계약(마스터).

```
evaluate(policy_IR, projection_IR, snapshot, revenue_events[], run_id)
  → { format, run_id, policy_digest, projection_digest, snapshot_digest, events_digest,
      bind_time, pool_lines[], lines[], holdings[], trace[], result_digest }
```

- 순수 함수. 입력 digest가 같으면 `result_digest`가 같다.
- `policy.depends_on`에 사실 참조와 digest가 있어야 하고 `projection.facts_digest`가 그것과 일치해야 한다 (`CORE_DIGEST_MISMATCH`).
- 이벤트 배열은 id 엄격 오름차순, 금액은 양의 정수 문자열, 유형·통화는 정책 scope 안.
- 출력 순서는 규범적으로 고정되어야 한다. 0원 line·holding은 만들지 않되 trace에는 남긴다.

벡터 함수 확장점. 마스터는 벡터 함수의 **시그니처 규약**만 정한다: 입력은 `(projection, snapshot, 인자)`, 출력은 `PartyVector`(합 1, `@hold` 허용), 인자의 비율은 bps 문자열. 합성 연산 `mix`(가중 합, Σw = 10000bps), `cap`, `floor_guarantee`(반복 수위 채우기, `ALLOC_INFEASIBLE`)는 마스터 표준 함수로 둔다. 잎 함수는 프로파일 등록값이다. 중첩 깊이 상한 64.

반올림. `rounding.method`는 필수. 마스터 레지스트리 값: `HIERARCHICAL_LRM_V1`. `tie`는 현재 `party_id_asc`만.

정적 검증(마스터 최소). 분할 층·split·mix 합 10000bps, 커버리지(투영에 권리자가 있는 역할마다 다루는 버킷 존재 — `ALLOC_UNCOVERED_CLAIMANT`), 통화·역할 타입 일치, 파이프 후 합 1, 실현 가능성.

프로파일이 정하는 것. 분할 층 구조, 잎 벡터 함수의 의미, 버킷 역할 추론, 실연 커버리지, 라인 역할 도출.

6.6 ledger 접점

마스터가 정하는 것. allocation 결과 IR의 `result_digest`와 `run_id`가 원장 기록의 참조 키다. 원장은 `lines[]`·`holdings[]`를 그대로 적재하며 수정하지 않는다. 보류 사유(`holding_reason`)는 확장점이며 마스터 등록값으로 `RIGHTS_DISPUTE | FRAUD_REVIEW`를 둔다. 정정은 새 `run_id`로 재평가하고 원장이 차액·역분개를 계산한다(계산 규격은 원장 몫).

마스터가 정하지 않는 것(본 버전 범위 밖). 역분개와 `line_seq` 증가 규칙, 지급 지시, 선급금 회수, 최소 보장, 다통화·환율, 세무 공제, 복식 원장 검증식. 언어로 승격할지는 8.4절.

어댑터 경계. DDEX ERN/DSR/RIN, CWR, ISWC/ISRC, ODRL 출력은 변환만 한다. 어댑터는 규칙을 판정하거나 금액을 재계산해서는 안 된다.

6.7 확장점 레지스트리

다음 표의 오른쪽 두 열은 **예시 프로파일**이다. 마스터 규범이 아니며, 특정 창작 모드가 확장점에 어떤 값을 등록할 수 있는지를 보이기 위한 스케치다.

확장점	마스터 정의(타입·불변식)	예시 프로파일: 보컬 트랙 재배치	예시 프로파일: AI 믹스
`asset_kind`	자산 종류. 구조 관계 `of`로 상위 자산 참조	`work`, `recording`, `voice`, `stem`(`of`, `slot`, `rights_bearing`)	`work`, `recording`, `stem_set`, `mix_session`, `model_run`
`edge_kind`	지분 엣지의 이름과 허용 (source, target) 종류	`PERFORMED_BY`, `OWNED_BY`, `CONTROLLED_BY`, `WRITTEN_BY`, `CONTRIBUTED_BY`	+ `MIXED_BY`, `MODEL_BY`
`claimant_role`	역할별 벡터의 키. 개수·이름 미정	`MASTER_OWNER`, `WORK_AUTHOR`, `CMO`, `PERFORMER_FEATURED`, `PERFORMER_NON_FEATURED`, `AGENCY`, `AI_PROVIDER`, `RECORDING_CONTRIBUTOR`	`MASTER_OWNER`, `WORK_AUTHOR`, `CMO`, `MIX_ENGINEER`, `MODEL_PROVIDER`
`party_kind`	당사자 종류. 위임 수령인 역할 규칙에 사용	`LABEL`, `PERFORMER`, `AUTHOR`, `PUBLISHER`, `CMO`, `AGENCY`, `AI_PROVIDER`, `SESSION`	+ `MODEL_OPERATOR`
`mandate_right`	위임 가능한 권리 종류	`MASTER`, `WORK`, `PERFORMER`	`MASTER`, `WORK`
`usage_metric`	사용량 `u`의 정의·단위·정규화	`ASSIGNED_LAYER_MS`(구성요소별 배정 시간, 레이어 합 허용, Σu = 1로 정규화)	세션 단위 고정 1 또는 이득 가중 시간
`vector_fn`(잎)	`PartyVector` 반환, 합 1	`graph(role)[via mandate]`, `contract(party)`, `member_pct()`, `performance_pool(mode,…)`	`graph`, `contract`, `model_share(run)`
`vector_fn`(합성, 마스터 표준)	`mix`, `cap`, `floor_guarantee`	동일	동일
`pool_mode`	프로파일 풀 함수의 모드	`USAGE_WEIGHTED`, `FIXED`, `HYBRID`, `MEMBER_PCT`	해당 없음
`event_type`	수익 이벤트 종류. 정책 scope의 원소	`PRIMARY`, `RESALE`, `TRAFFIC`	`PRIMARY`, `SUBSCRIPTION_SHARE`

확장점	마스터 정의(타입·불변식)	예시 프로파일: 보컬 트랙 재배치	예시 프로파일: AI 믹스
`binding_time`	정산에 쓸 `as_of` 결정 규칙 이름	`ISSUE_TIME`, `USAGE_TIME`, `SALE_TIME`, `REPORT_PERIOD`	동일(마스터 후보)
`rounding_method`	층별 보존 반올림 알고리즘	`HIERARCHICAL_LRM_V1`	동일(마스터 값)
`holding_reason`	보류 사유	`RIGHTS_DISPUTE`, `FRAUD_REVIEW`	+ `MODEL_LICENSE_PENDING`
`action`	이용 행위 코드 (Scope.actions 원소)	재생·소장·숏폼 등 + `AI_TRAIN`, `AI_GENERATE`	동일 + `MODEL_INFER`
`decision`	허락 판정 결과	`PERMIT`, `PERMIT_WITH_OBLIGATION`, `DENY`, `REVIEW_REQUIRED`	동일(마스터 값)
분할 층 구조	마스터는 층 수·이름을 정하지 않음	**예시 프로파일**: Tier1 3풀(`platform`/`ops`/`rights`) + Tier2 권리자 풀 내부 split. `pack`은 정책 scope 필드	세션 단위 1층 또는 모델 대가/권리자 2층
프로파일 전용 모듈	rank 2·3에 추가 가능	조합 제약·승인 체인·유일성 범위 `PACK`	없거나 `model_policy`

마스터 값으로 승격 후보: `binding_time`, `rounding_method`, `decision`, `visibility`, 마스터 합성 벡터 함수. 확정은 두 번째 프로파일 작성 시(8.3절).

AI 믹스 열은 "특정 자산 모델을 강제하지 않아도 같은 코어로 표현 가능함"을 보이기 위한 스케치다. 실제 역할·계약 구조는 학습 데이터·모델·목소리 이용·출력물 각각의 허락을 분리하는 원칙을 따라 별도 프로파일로 설계해야 한다.

6.8 표면 문법 스케치 (EBNF)

본 초안이 정하는 **중립 표면 문법**이다. 프로파일은 같은 IR로 컴파일되는 한 별칭을 추가할 수 있으나, 코어 키워드를 다른 의미로 재정의해서는 안 된다.

```
document        = allocation_doc | facts_doc ;
allocation_doc  = "allocation" string "version" integer "{" alloc_body "}" ;
alloc_body      = uses_stmt scope_stmt bind_stmt layer_stmt+ rounding_stmt ;
uses_stmt       = "uses" "facts" string "@" integer "digest" string ";" ;
scope_stmt      = "scope" "{" scope_field+ "}" ;
scope_field     = ident ":" ( ident | ident_list | ident ) ";" ;
bind_stmt       = "bind" "{" "time" ":" ident ";" "}" ;
layer_stmt      = "layer" integer [ ident ] "{" layer_body "}" ;
layer_body      = named_bps+ | "split" "{" split_item+ "}" ;
named_bps       = ident "=" ( percent | bps_lit ) ";" ;
split_item      = ident "=" vector_expr ":" ( percent | bps_lit ) ";" ;
vector_expr     = ident "(" arg_list? ")" [ "via" "mandate" ] ;
rounding_stmt   = "rounding" "{" "method" ":" ident ";" "tie" ":" ident ";" "}" ;
percent         = integer "%" | integer "." integer "%" ;
bps_lit         = integer "bps" ;
ident_list      = "[" ident ( "," ident )* "]" ;
```

`layer N` 은 분할 층이다. 층 이름·버킷 이름은 프로파일 어휘다. `guard`/`exceptions` 블록은 엔진 고정 의미의 문서화로 둘 수 있으나 IR에 남지 않으며, 마스터가 문법을 규정하지 않는다.

6.9 예시

모든 값은 예시다.

(a) 코어 모델만으로 표현한 최소 사실 주장 (제품 어휘 없음)

```
{
  "lang": "1.0",
  "module": "facts",
  "id": "example-core-only",
  "version": "1",
```

```
  "depends_on": [],
  "visibility_default": "PARTIES",
  "body": {
    "parties": [
      { "id": "pty_a", "kind": "AUTHOR" },
      { "id": "pty_b", "kind": "AUTHOR" },
      { "id": "pty_c", "kind": "CMO" }
    ],
    "assets": [
      { "id": "ast_work_1", "kind": "work" }
    ],
    "claims": [
      { "id": "c1", "source": "ast_work_1", "edge": "WRITTEN_BY", "target": "pty_a",
        "share": "6000", "status": "VERIFIED",
        "valid": { "from": "2026-01-01T00:00:00.000Z" },
        "recorded_at": "2026-01-01T00:00:00.000Z" },
      { "id": "c2", "source": "ast_work_1", "edge": "WRITTEN_BY", "target": "pty_b",
        "share": "4000", "status": "VERIFIED",
        "valid": { "from": "2026-01-01T00:00:00.000Z" },
        "recorded_at": "2026-01-01T00:00:00.000Z" }
    ],
    "mandates": [
      { "id": "m1", "from": "pty_b", "to": "pty_c", "right": "WORK", "share": "10000",
        "status": "VERIFIED",
        "valid": { "from": "2026-01-01T00:00:00.000Z" },
        "recorded_at": "2026-01-01T00:00:00.000Z" }
    ]
  }
}
```

이 문서는 "작품 하나에 두 저작자가 60/40이고, B의 몫은 단체 C가 수령한다"만 말한다. 계산식·정책·특정 창작 모드 개념이 없다. 여기에 `(as_of, known_at)`과 `u = {ast_work_1: 1}`을 주면 `WORK_AUTHOR` 벡터 `{pty_a: 3/5, pty_b: 2/5}`가, `via mandate` 라우팅 후에는 `{pty_a: 3/5, pty_c: 2/5}`(pty_c의 라인 역할은 `CMO`)가 나온다.

(b) 안분 정책 — 중립 표면 문법

```
allocation "ALLOC-EXAMPLE-01" version 1 {
  uses facts "work-facts-01"@1 digest
"sha256:4c8fc5573e79e0da0e23eee13566d4af8c5aa5fd319319f7b2684a78077ca6f4";
  scope { catalog: cat_demo_01; events: [PRIMARY, RESALE]; currency: KRW; }
  bind { time: ISSUE_TIME; }
  layer 1 {
    platform = 25%;
    ops = 500bps;
    rights = 70%;
  }
  layer 2 rights {
    split {
      master = graph(MASTER_OWNER) : 45%;
      performer = mix(fixed: 40%, usage: 60%) : 45%;
      work = graph(WORK_AUTHOR) via mandate : 10%;
    }
  }
  rounding { method: HIERARCHICAL_LRM_V1; tie: party_id_asc; }
}
```

코어 어휘: `uses … digest`, `bind.time`, `%`/`bps`, `via mandate`, `rounding.method`, `layer`. `catalog`·버킷 이름·잎 함수 인자는 프로파일 또는 예시 값이다.

(c) 위 (b)의 JSON IR

```
{
  "lang": "1.0",
```

```
  "module": "allocation",
  "id": "ALLOC-EXAMPLE-01",
  "version": "1",
  "depends_on": [
    { "ref": "rl:facts/work-facts-01@1",
      "digest": "sha256:4c8fc5573e79e0da0e23eee13566d4af8c5aa5fd319319f7b2684a78077ca6f4" }
  ],
  "visibility_default": "PARTIES",
  "body": {
    "scope": { "catalog": "cat_demo_01", "events": ["PRIMARY", "RESALE"], "currency": "KRW" },
    "bind": { "time": "ISSUE_TIME" },
    "layers": [
      {
        "index": "1",
        "buckets": [
          { "name": "platform", "bps": "2500" },
          { "name": "ops", "bps": "500" },
          { "name": "rights", "bps": "7000" }
        ],
        "pass_to": "rights"
      },
      {
        "index": "2",
        "of": "rights",
        "split": [
          { "name": "master", "bps": "4500", "role": "MASTER_OWNER",
            "vector": { "fn": "graph", "role": "MASTER_OWNER" } },
          { "name": "performer", "bps": "4500", "role": "PERFORMER",
            "vector": { "fn": "mix", "fixed": "4000", "usage": "6000" } },
          { "name": "work", "bps": "1000", "role": "WORK_AUTHOR",
            "vector": { "fn": "graph", "role": "WORK_AUTHOR", "via_mandate": true } }
        ]
      }
    ],
    "rounding": { "method": "HIERARCHICAL_LRM_V1", "tie": "party_id_asc" }
  }
}
```

같은 입력에 대한 기대 결과 모양: `format: "allocation-result/1"`, 1층 `pool_lines`(예: 10,000원 → 2,500/500/7,000; 1,001원 → 250/50/701, 잔여 1원은 소수부가 큰 `rights`에 감), 역할별 `lines`, `result_digest`.

(d) 투영 결과 IR의 모양 (마스터 주장 분쟁)

```
{
  "format": "projection/1",
  "as_of": "2026-09-10T00:00:00.000Z",
  "known_at": "2026-09-10T00:00:00.000Z",
  "vectors": {
    "MASTER_OWNER": { "@hold": "1/1" },
    "PERFORMER": { "pty_p1": "2/3", "pty_p3": "1/3" },
    "WORK_AUTHOR": { "pty_a": "3/5", "pty_b": "2/5" }
  },
  "paths": [
    { "asset": "ast_p1", "role": "PERFORMER", "party": "pty_p1",
      "weight": "1/1", "claims": ["c1", "c3"] }
  ],
  "facts_digest": "sha256:…",
  "u_digest": "sha256:…",
  "digest": "sha256:a6139dcd…"
}
```

`MASTER_OWNER` 벡터 전체가 `@hold`이고 다른 역할은 영향받지 않는다(5.6).

6.10 프로파일 작성 규칙

새 프로파일 문서는 다음을 포함해야 한다(MUST).

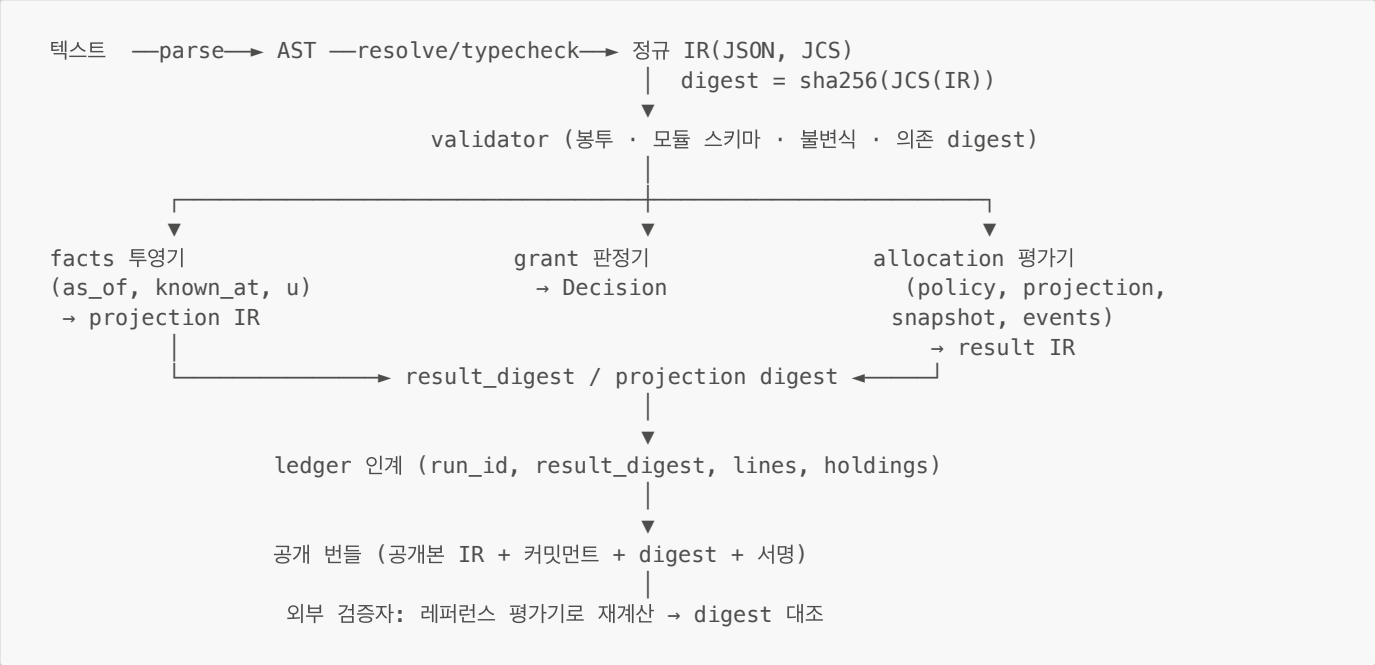
1. 대상 마스터 버전.
2. 확장점별 등록값과 각 값의 정의(6.7 표의 자기 열).
3. 자산 인코딩(마스터 추상 모델 → 구체 배열/필드)과 JSON Schema.
4. 역할 경로 정의(엣지 연쇄 → 역할).
5. 사용량 메트릭 정의와 정규화 규칙.
6. 분할 층 구조와 잎 벡터 함수의 의미.
7. 프로파일 전용 모듈(있다면)과 그 rank, 오류 코드 접두.
8. 코어 불변식에 대한 **강화** 목록(완화는 불가).
9. 적합성 벡터(입력·기대 결과·기대 digest) 최소 1세트/모듈.
10. 정본 결정 표(선행 운영 관행과의 충돌 해소).

---

7. 시스템 구현

이 장은 준수 구현이 따라야 하는 **참조 아키텍처와 적합성 프로그램**을 정한다. 특정 저장소·특정 제품의 구현 현황·시험 개수는 본 규격의 대상이 아니다.

7.1 참조 아키텍처



[N-7.1] 평가·투영·검증은 순수 함수여야 한다. 운영 정산 엔진은 레퍼런스 평가기와 **같은 의미론**을 사용해야 한다. 검증용 구현과 지급용 구현을 갈라 다른 결과를 내는 것은 금지한다.

[N-7.2] 파서·포매터를 두는 구현은 왕복 불변을 만족해야 한다: 같은 IR에 대해 `parse(format(ir)) = ir` (공백·주석·선언 순서 제외).

권고 구현 선택: 런타임 의존성을 최소화하고, 중간 계산은 언어 네이티브 큰 정수(또는 동등한 정확 유리수)로 수행한다. 해시·서명은 결정적 입력에만 의존해야 한다. 브라우저와 서버가 같은 라이브러리를 쓰는 것이 좋으나(SHOULD), 필수 요건은 digest 일치다.

명령줄 또는 동등한 인터페이스를 두는 구현은 다음을 제공하는 것이 좋다(SHOULD): `compile`, `check`, `project`, `evaluate`, `verify`. 결과 IR은 표준 출력, 진단은 표준 오류에 `<CODE> <path> <message>` 한 줄씩. 종료 코드는 성공 0, 검증 실패 1, 사용 오류 2를 권고한다. `evaluate`와 `verify`는 호출자가 넣은 투영을 신뢰하지 않고 입력으로 다시 계산해야 한다(MUST, 제공할 경우).

7.2 적합성 수준

수준	포함	통과 조건
Level 0 — core + facts	봉투·타입·JCS digest·레지스트리·오류 모델 · 사실 검증·이중 시간축·투영·위임·`@hold`·활성화 판정·공개본 검증	사실 적합성 벡터 전건에서 투영 IR과 `digest` 일치. `CORE_*`, `FACTS_*` 오류 코드 전건 재현
Level 1 — + allocation	정적 검증·벡터 함수·평가기·계층 최대잔여·결과 검증	안분 적합성 벡터 전건에서 `result_digest` 일치. 속성 시험(금액 보존·결정성) 통과. 텍스트 컴파일 결과가 IR과 동일

수준	포함	통과 조건
Level 2 — + grant	요구사항 AND/OR, meet-only Scope, 닫힌 거절, 철회 epoch	grant 적합성 벡터 전건에서 `Decision`과 이유 코드 일치. 범위 확대는 오류
Level 3 — + 원장 접점·공개 번들	`run_id`/`result_digest` 인계, 보류 적재, 공개본 3단계 검증(구조·불변식·재계산)	원장 적재 후 라인·보류가 결과 IR과 비트 단위로 같고, 공개 번들 재계산 digest가 일치

프로파일 적합성은 수준과 별개로 "프로파일 X의 적합성 벡터 통과"로 표시한다. 한 구현이 Level 1을 주장하려면 마스터 벡터와 자기가 선언한 프로파일 벡터를 모두 통과해야 한다.

본 규격의 코어와 안분 층은 보컬 조합 프로파일의 레퍼런스 구현으로 개념 검증되었다.

7.3 적합성 벡터와 골든 digest

[N-7.3] 적합성 벡터는 최소 `{ input, expected }` 쌍이어야 한다. `input`은 사실·정책·스냅샷·이벤트·시점·선언 청구권자를 포함하고, `expected`는 투영 IR 또는 결과 IR 전체(digest 포함)여야 한다.

[N-7.4] 기대값은 **평가기 결과 자체**여야 한다. 형식 태그 도입처럼 digest만 바뀌는 변경에서도 금액·라인·보류가 그대로임을 대조할 수 있어야 한다.

권고 시험 종류:

- **속성 시험**: 무작위 변형 입력에 대해 `Σ amount = R`, 벡터 합 1, 같은 입력 → 같은 digest, 공백·선언 순서 불변성.
- **오류 시험**: 예약된 오류 코드별 최소 재현. 도달 불가 방어 코드(`ALLOC_ROUNDING_INVARIANT` 등)는 이유와 함께 예외로 둘 수 있다.
- **회귀**: 본 문서 2.4절·6.9절의 계산 예시를 골든 케이스로 편입하고, 예시가 틀리면 문서를 정정한다.

새 프로파일은 최소 모듈당 1세트의 적합성 벡터를 제출해야 한다(6.10 규칙 9).

7.4 플랫폼 통합 패턴

준수 구현이 기존 운영 시스템과 붙을 때의 **방향**을 정한다. 저장 스키마 자체는 본 규격의 대상이 아니다.

통합점	플랫폼 측	언어 측	방향
권리 사실 저장소	당사자·작품·녹음·크레딧·위임 행	facts IR. 기존 행을 갱신하지 않고 새 버전 행을 추가	IR → 저장소(컴파일 산출) 또는 저장소 → IR(등록)
동의·허락 게이트	동의 계층, 이용허락, 지역·행위·부분 범위	grant 층 입력(범위·등급·유효기간). 승인·직무 분리는 프로파일 모듈	저장소 → IR(사실 등록), IR → 게이트(판정)
정산 정책 편집	폼·슬라이더·직접 편집	텍스트 ↔ IR 왕복. `PARSE_*` 진단은 줄·열을 포함하는 것이 좋다	화면 ↔ 텍스트 ↔ IR
발행 스냅샷	발행 시 동결된 멤버 비율·사용량	평가기 입력 `snapshot`, 사용량 `u`	저장소 → 평가기
수익 이벤트	이용·판매 행, 무결성 상태	`RevenueEvent.integrity ∈ {CLEAR, FLAGGED}` → `FRAUD_REVIEW` 보류	저장소 → 평가기
정산 원장	분배 라인·보류 계정	결과 IR `pool_lines`, `lines`, `holdings`. 유일키 `(run_id, event_id, party_id, role, bucket, line_seq)`	평가기 → 원장
정책 거버넌스	초안·컴파일·승인·활성·대체	서명 역할(편집자·승인자·활성화자), 직무 분리	양방향
감사·인스펙터	추적 읽기 모형	결과 IR `trace[]`, 투영 `paths[]`	평가기 → 읽기 모형
식별자·공급망 어댑터	ISWC/ISRC, CWR, DDEX	변환만. 판정·계산 금지	양방향 변환

[N-7.5] 게시된 IR을 운영 행이 직접 갱신해서는 안 된다. 변경은 새 `(id, version)` 문서와 새 봉인이다.

[N-7.6] 원장은 결과 IR의 라인·보류를 수정 없이 적재해야 한다. 차액·역분개는 새 `run_id`의 결과에 대해서만 계산한다.

7.5 운영

늦게 도착한 사실의 재계산. 새 사실 버전(`facts@v+1`)이 게시되면 영향 결과물을 역방향 인덱스로 찾아 정책의 `bind.time` 규칙대로 `(as_of, known_at)`을 정해 재투영·재평가한다. 결과는 새 `run_id`이며 원 결과는 그대로 남는다. 차액·역분개는 원장이 계산한다. 어떤 이벤트를 재계산 대상으로 삼을지(발행 시점 고정 vs 이용 시점 기준)는 정책이 `bind.time`으로 선언하며 언어 기본값은 없다.

감사 추적. 결과 IR의 `trace[]`와 `paths[]`가 라인마다 "어느 주장(claim id) → 어느 벡터 → 어느 층에서 얼마"를 남긴다. 공개 목록에는 각 부분의 `(module, id, version, digest, public_digest)`와 서명이 있어야 한다. 장기 감사 로그는 이 digest들을 참조 키로 쓰는 것이 좋다(SHOULD).

공개. 공개본은 `PUBLIC`이 아닌 값을 커밋먼트로 치환한다. salt 전달·키 관리 방식은 구현 몫이다. 검증 인터페이스는 같은 평가기로 재계산해야 한다.

성능. 투영은 역할별 행렬 `W_role = u · M_role`이며 사실 문서가 같으면 `M_role`을 재사용할 수 있다. 본 규격은 성능을 규정하지 않는다.

보안. id는 `^[A-Za-z0-9._-]+$`, `__proto__` 금지, 소유 속성만 조회, JSON 깊이 256, 알 수 없는 키 거부, 텍스트 표시 시 제어·양방향 문자 이스케이프.

7.6 구현 범위 요약

본 버전에서 코어로 확정되어 준수 구현이 제공해야 하는 것(Level 1 주장 시): 봉투·타입·JCS digest·레지스트리·오류 모델, facts 검증·이중 시간축·투영·위임·`@hold`·활성화 판정·공개본 검증, allocation 정적 검증·표준 합성 함수·평가기·`HIERARCHICAL_LRM_V1`·결과 검증, allocation 텍스트 문법 또는 동등한 IR 입력.

방향 확정(문법 미고정) 또는 범위 밖: grant·constraint·approval 모듈의 표면 문법, Scope meet 연산 IR, facts 텍스트 입력 문법, 공개 번들 포털, DDEX/ODRL 어댑터, 원장(역분개·다통화·최소보장·회수), 분산원장 고정, 마스터 수준 추상 인코딩의 필수 여부, 코어/프로파일 레지스트리의 물리적 분리.

---

8. 응용분야 및 권리 문제 해소 효과

8.1 창작 모드별 응용

다음 표의 "프로파일이 등록할 어휘" 열은 **예시 프로파일**이다. 마스터는 어느 행의 자산 모델도 강제하지 않는다.

창작 모드	예시 프로파일 어휘	공통(마스터 코어·표준 함수)	특유의 불변식 후보
보컬 트랙 재배치	`stem`/`slot`/`pack`, 역할 집합, `ASSIGNED_LAYER_MS`, `member_pct`, `performance_pool`, `PRIMARY`/`RESALE`/`TRAFFIC`, Tier1 3풀, 조합 제약	봉투·digest·이중 시간축·투영·`@hold`·계층 최대잔여·역할별 라인	이용자 비율은 실연 풀 안에서만(원저작자 몫 불변), 분쟁 시 묶음 활성화 금지
보컬 트랙 추가 녹음	`recording{of: work, derived_from?: recording}`, 녹음별 `MASTER_OWNER` 분리, 신규 실연 `contract`	동일	새 녹음 마스터는 원 마스터를 승계하지 않음 → 별 `CONTROLLED_BY` 그룹
AI 믹스/마스터링	`mix_session`, `model_run`, `MIX_ENGINEER`, `MODEL_PROVIDER`, 세션 단위 사용량	동일	모델 대가는 계약 벡터로만(사용량 기반 실연료와 동일시 금지), `AI_GENERATE` 행위 허락 필수
AI 보이스 실연	`voice{kind: AI, source_voice}`, `AI_PROVIDER`, `MODEL_OPERATOR`, `AI_TRAIN`/`AI_GENERATE` 분리	동일	원천 음성 제공자·모델 운영자 각각 허락, 철회 시 신규 생성 차단, 생성물 표시 의무
커버/리믹스	`work` 중심, `arrangement` 자산, `ARRANGER`, 원 녹음 사용 유무	동일	작품 허락 범위에 편곡 포함 여부, 원 녹음 미사용 시 마스터 경로 없음
샘플링	`sample_use{of: recording, from_ms, to_ms}`, 곡별 `mix(...)`	동일	원 녹음 + 원 작품 모두 허락 AND, 곡별 벡터 가중치 승인
이용자 지정 비율 묶음	유일성 범위, `member_pct`	동일	이용자 지정 비율은 승인된 실연 풀 안에서만

모든 행이 공유하는 것은 코어(5.1·5.2·5.3·5.5·5.6·5.7·5.8)와 표준 합성 함수다. 행마다 다른 것은 자산·역할·잎 함수·이벤트 어휘다.

8.2 문제 → 효과 매핑

1·2장의 문제	규격 요소	기대 효과	성숙도
미매칭·늦은 정산 (1.6.1, 1.6.3, P5)	이중 시간축 + 새 사실 버전 재투영 + `bind.time`	늦게 매칭된 사실을 소급 재계산하되 원 결과도 재현. "당시 왜 그렇게 계산했나"와 "지금 기준 얼마가 맞나"를 둘 다 답함	확정(재계산 워크플로는 운영)

1·2장의 문제	규격 요소	기대 효과	성숙도
지분 분쟁(1.6.2, P6)	그룹 `@hold`, 재정규화 금지, `FACTS_DISPUTE_OPEN`	분쟁 몫만 보류, 타인 몫 불변, 분쟁 중 새 결과물 활성화 차단	확정
허락 범위 불명(1.6.4, P4)	meet-only Scope, 축소 전용 계승, `GRANT_SCOPE_WIDENING`	n차 결과물의 허용 범위가 재료 범위의 교집합으로 자동 도출, 확대 불가	방향 확정(문법 미고정)
불투명 정산(1.6.3)	JCS digest 체인, 공개 등급·커밋먼트, 레퍼런스 평가기 재계산	제3자는 구조·불변식·digest, 당사자는 자기 몫 재계산. 운영 엔진 = 레퍼런스 엔진	확정(전달·포털은 구현)
역할 혼동·세무 구분(P2)	역할별 벡터·라인	같은 사람의 작곡·실연 몫이 끝까지 분리, 위임 수령도 귀속/수령 분리	확정
반올림 불일치·소액 누출(P3)	`HIERARCHICAL_LRM_V1`, 층별 보존 검증, 방법 이름 고정	1 최소 단위까지 보존, 운영 코드마다 다른 잔여 귀속 재발 방지	확정
위임 구조 복잡성(1.2)	`mandate` ≤ 2홉, 권리 종류별	신탁 여부 불리언이 잃던 종류·기간·지분 차이를 표현	확정
플랫폼 고유 역할(P7)	확장점 레지스트리	팬 기여자·AI 제공자를 코어 변경 없이 등록	확정(물리 분리는 방향)
안분 = 허락 오해(P8, 1.6.4)	층 분리, 닫힌 거절, 별 오류 네임스페이스	계산 결과가 허락을 대체할 수 없음이 구조화	코어 확정, grant 판정은 방향
AI 귀속 불확실(1.6.5)	언어는 귀속을 판정하지 않음 + 프로파일 어휘	기여 사실과 계약 대가만 기록·계산, 법적 판단은 밖에 둠	원칙 확정, AI 프로파일은 별도
부정 이용(1.6.6)	`integrity = FLAGGED` → `FRAUD_REVIEW` 보류	판정 전 격리를 결정적으로 처리, 확정 시 원장이 역분개	확정
식별자·보고 단절(1.3)	어댑터 경계, 사용 관계는 사실 문서	ISWC/ISRC/CWR/DDEX를 대체하지 않고 계산 층을 얹음	확정

8.3 단계별 로드맵

단계	산출물	완료 기준
v0.2 (본 문서)	마스터 초안: 현행 체계 대비 문제 제기, 코어 경계, 확장점, 중립 문법·IR, 적합성 수준	검토. 부록의 미결 항목 해소
v0.3	grant 층 문법·IR 스키마·오류 코드(`GRANT_*`), `Scope` 타입과 meet 연산, Level 2 적합성 벡터. 코어/프로파일 레지스트리 물리 분리 설계	grant 판정기가 grant 적합성 벡터를 통과. 동의·허락 저장 모형과의 매핑표 확정
v0.4	두 번째 프로파일(후보: AI 믹스 또는 보컬 트랙 추가 녹음) 초안. 마스터 승격 후보(`binding_time`, `rounding_method`, `decision`, 합성 함수) 확정. 마스터 수준 자산 인코딩 결정	두 프로파일이 같은 코어 라이브러리로 Level 1 통과
v1.0	첫 두 프로파일 발행, 원장 접점 문법 결정, 공개 번들 목록 규격, 적합성 스위트 공개	외부 구현 1개 이상이 Level 1 적합성 통과. 변리사 검토 결과 반영

8.4 미결 질문

#	질문	영향	결정 시점
Q1	봉투 필드 `lang`을 확정 영문 가칭 이름으로 바꾸는가, 프로파일 식별 필드(`profile`)를 추가하는가	모든 IR digest가 바뀜(MAJOR)	v1.0
Q2	마스터 수준 추상 자산 인코딩(6.9(a))을 코어가 받는가, 프로파일 인코딩만 존재하고 마스터는 추상 모델 문장으로만 남는가	코어 검증기 범위, 프로파일 간 사실 문서 이식성	v0.4
Q3	동의(`CONSENT`)의 범위를 grant 층에, 승인(`APPROVAL`)을 프로파일 모듈에 두는 6.4의 권고를 채택하는가	grant IR 스키마	v0.3
Q4	원장 문법(역분개·`line_seq`·최소보장·회수·다통화)을 언어로 승격하는가, 원장 구현에 맡기는가	층 수, 적합성 수준	v1.0
Q5	위임 홉 상한 2를 마스터 상수로 고정하는가, 프로파일 파라미터(상한 ≤ 마스터 최대)로 두는가	`FACTS_MANDATE_DEPTH` 의미	v0.3

#	질문	영향	결정 시점
Q6	5.12 결합 후보에 대한 선행기술 조사·청구항 초안	출원 여부	사업 결정
Q7	`guard`/`exceptions` 블록을 마스터가 규정하는가(현재 IR에 없음)	텍스트 문법 이식성	v0.4
Q8	서명·해시의 브라우저와 서버 구현을 동일 라이브러리로 고정하는가	로컬 검증 가능 여부	구현 계획
Q9	`AI_TRAIN`/`AI_GENERATE`를 마스터 `action` 값으로 승격하는가	AI 계열 프로파일 공통성	v0.4
Q10	첫 상용 계약에서 확정해야 하는 요율·풀 비율·이벤트 정의의 데이터 확보 책임	정책 활성화의 입력 조건(개발로 종결 불가)	사업 결정

부록 A. 용어집

한국어	English	정의
n차 창작	n-th order creation	1차 원저작물·원녹음 → 2차 이용 → 3차 이상 재이용의 연쇄(2.1)
2차 이용	secondary use	1차 결과물의 구성요소를 허락 범위 안에서 재사용해 새 결과물을 만드는 행위. 저작물 성립 판단과 구분
마스터 규격	master specification	언어 패밀리의 공통 코어와 확장점을 정하는 최상위 문서(본 문서)
프로파일	profile	확장점에 값을 등록하고 자산 인코딩·분할 층 구조를 정하는 응용 규격
코어	core	프로파일과 무관한 타입·봉투·digest·오류 모델·불변식
확장점	extension point	마스터가 이름·타입·불변식만 정하고 값은 프로파일이 등록하는 자리(6.7)
권리 사실	rights fact	계산식 없는 주장(claim)·위임(mandate) 선언. facts 층
주장	claim	`(source, edge, target, share, valid, recorded_at, status)`의 지분 엣지
위임	mandate	위임인이 받을 몫 중 수임인이 수령하는 비율. 권리 종류별, ≤ 2홉
이중 시간축	dual time axis / bitemporal	효력 구간 `valid`와 기록 구간 `recorded_at/superseded_at`. 질의 `(as_of, known_at)`
투영	projection	사실 문서와 사용량 벡터로 역할별 `PartyVector`를 계산하는 순수 함수
기여 벡터 / 당사자 벡터	contribution vector / PartyVector	역할별 `Map<PartyRef, Ratio>`, 합 1. 임베딩 벡터와 무관
청구권자 역할	claimant role	벡터의 역할 키(프로파일 등록값)
보류 키	hold key (`@hold`)	분쟁 그룹 몫이 벡터에 남는 특수 키. 청구권자 아님
이용허락 / 게이트	grant / gate	요구사항 AND·대체 허락 OR로 `PERMIT/…/DENY`를 내는 층. 닫힌 거절
범위	Scope	`{actions, territories, media, period, audience}` 격자. meet만 허용
meet / join	meet / join	격자의 교집합 / 합집합. 허락 판정에는 meet만
안분 정책	allocation policy	분할 층·버킷·벡터 식·반올림을 선언한 문서
평가기	evaluator	`evaluate(policy, projection, snapshot, events) → result`의 순수 함수
계층형 최대잔여	HIERARCHICAL_LRM_V1	층마다 floor 후 잔여를 소수부 내림차순·키 오름차순으로 배분, 층별 보존 검증
금액 보존	money conservation	모든 분할 층에서 Σ 하위 = 상위
결과 digest	result_digest	결과 IR(형식 태그 포함)의 JCS digest
문서 봉투	envelope	`{lang, module, id, version, depends_on, visibility_default, body}`

한국어	English	정의
의존 고정	dependency pinning	`depends_on[].digest`로 참조 대상의 내용을 고정
JCS	JSON Canonicalization Scheme (RFC 8785)	결정적 JSON 직렬화
공개 등급	visibility	`PUBLIC | PARTIES | PRIVATE`
커밋먼트	commitment	`sha256(salt ‖ JCS(value))`. 비공개 값의 공개본 치환
증명 서명	attestation	`{signer, role, over, alg, sig}`. 서명 메시지에 역할·서명자 포함
직무 분리	SoD (separation of duties)	편집자와 활성화자 구분 등
적합성 수준	conformance level	Level 0~3(7.2)
골든 digest	golden digest	적합성 벡터의 기대 digest
사용량 벡터	usage vector (`u`)	결과물별 자산 사용량. 메트릭은 프로파일
스냅샷	snapshot	발행 시 동결된 정산 입력. 프로파일이 내용을 정함
닫힌 거절	fail-closed	판정 불가 = 거부

부록 B. 최소 참조

본 문서는 최상위 규격이며 상위 문서가 없다. 준수에 필요한 외부 규범과, 본 규격이 **대체하지 않고 경계로 두는** 산업 표준만 적는다. 프로파일 규격·구현·저장 스키마는 각자의 문서가 본 문서를 대상으로 선언한다.

구분	문서	본 규격에서의 역할
규범 의존	RFC 8785 JSON Canonicalization Scheme	IR 직렬화·digest(5.8, 6.2.3)
법적 배경	대한민국 저작권법 제5조·제22조·제46조·제105조	1차 권리·이용허락·신탁의 전제(1.1, 1.2). 본 규격은 법률이 아니다
식별자 경계	ISO 15707 (ISWC), ISO 3901 (ISRC)	작품·녹음 식별. 사용 관계는 본 규격의 사실 층
운반 경계	CWR (CISAC), DDEX ERN/DSR/RIN	사실·거래 운반. 판정·안분 없음
허락 어댑터	ODRL 2.2 (W3C)	grant 출력의 단방향 대상. 안분·시간축·digest는 손실
시간 모형 참고	SQL:2011 application-time / system-time	이중 시간축의 선행 개념. 문서 봉투·digest와의 결합은 본 규격

프로파일 문서가 본 마스터를 인용할 때는 대상 버전(`v0.2` 등)을 명시해야 한다.

부록 C. 외부 출처 (2026-09-29 확인)

주제	출처	URL
2차적저작물(제5조)	국가법령정보센터	https://law.go.kr/lsLinkCommonInfo.do?lsJoLnkSeq=1021872399
2차적저작물작성권(제22조)	국가법령정보센터	https://www.law.go.kr/lsLinkCommonInfo.do?chrClsCd=010202&lsJoLnkSeq=1033063597
이용허락(제46조)	국가법령정보센터	https://www.law.go.kr/LSW/lsLinkCommonInfo.do?chrClsCd=010202&lsJoLnkSeq=1029423517
저작권위탁관리업 허가(제105조)	국가법령정보센터	https://www.law.go.kr/lsLawLinkInfo.do?chrClsCd=010202&lsJoLnkSeq=1000979131
신탁관리단체 규정 목록 (KOMCA·KOSCAP·FKMP·RIAK 포함)	한국저작권위원회	https://www.copyright.or.kr/information-materials/trust-management/list.do
음악 권리별 신탁관리단체 안내	저작권비즈니스지원센터	https://findcopyright.or.kr/user/guide/srvcInfo/info.do
ISWC — ISO 15707:2022	ISO	https://www.iso.org/standard/83125.html
ISRC — ISO 3901:2019	ISO	https://www.iso.org/standard/64817.html
ISRC 표준·등록기관	IFPI ISRC	https://isrc.ifpi.org/isrc-standard

주제	출처	URL
ISRC Handbook	IFPI	https://isrc.ifpi.org/images/downloads/ISRC_Handbook.pdf
ISNI·IPI 개요(2차 자료)	Session Help Center	https://support.sessionstudio.com/en/articles/7053006-music-industry-identifiers
CWR 개요	CISAC	https://www.cisac.org/formats
CWR 사용 안내	The MLC	https://help.themlc.com/en/support/what-is-cwr
DDEX 표준 개요 (ERN·MEAD·DSR·RIN)	DDEX Knowledge Base	https://kb.ddex.net/about-ddex-standards/ddex-standards/
DDEX 표준 목록	DDEX	https://new.ddex.net/standards/
DDEX RIN FAQ	DDEX	https://kb.ddex.net/implementing-each-standard/recording-information-notification-%28rin%29/rin-faq/
DDEX MWN	DDEX	https://service.ddex.net/doc/Standards/MWN-P1-131/MWDR-3218%20-%20MWN%20Part%201%20Messages%20v1.3.1.pdf
The MLC 라이선스 범위	The MLC	https://blog.themlc.com/resources/what-types-of-licenses-does-the-mlc-administer
The MLC 미매칭·시장점유율 분배(역사적 미매칭 약 $397M, 매칭 >$317M, 분배 >$229M, 2027 시작 계획)	The MLC	https://www.themlc.com/marketshare
The MLC Historical Royalties	The MLC	https://www.themlc.com/historical-royalties
The MLC Blanket Royalties	The MLC	https://www.themlc.com/blanket-royalties
SoundExchange 디지털 실연 분배 구조	SoundExchange	https://www.soundexchange.com/digital-performance-royalties/
IFPI 스트리밍 무결성 공동 약속(2026-09-14)	IFPI	https://www.ifpi.org/music-companies-unite-in-new-commitment-to-combat-streaming-fraud-in-order-to-protect-artists-songwriters-and-fans/
IFPI AI 차트 적격성 원칙(2026-07)	IFPI	https://www.ifpi.org/ifpi-rolls-out-global-principles-for-the-eligibility-of-recordings-developed-using-ai-in-official-music-charts-worldwide/
미국 저작권청 AI 보고서 Part 2(2025-01-29)	U.S. Copyright Office	https://www.copyright.gov/newsnet/2025/1060.html
ODRL Information Model 2.2	W3C	https://www.w3.org/TR/odrl-model/
ODRL Vocabulary & Expression 2.2	W3C	https://www.w3.org/TR/odrl-vocab/
ODRL Formal Semantics	W3C ODRL CG	https://w3c.github.io/odrl/formal-semantics/
ccREL(W3C Member Submission, 2008)	W3C	https://www.w3.org/submissions/2008/02/
MPEG-21 REL(ISO/IEC 21000-5 CD 텍스트)	ISO/IEC JTC1/SC29 via coverpages	https://xml.coverpages.org/MPEG-REL-CD4942.pdf
RFC 8785 JCS	IETF	https://www.rfc-editor.org/rfc/rfc8785.html
SQL:2011 temporal features(bitemporal)	SIGMOD Record 2012	https://sigmodrecord.org/publications/sigmodRecord/1209/pdfs/07.industry.kulkarni.pdf
Temporal database(valid/transaction time)	Wikipedia	https://en.wikipedia.org/wiki/Temporal_database
Largest remainder / Hamilton method	Wikipedia	https://en.wikipedia.org/wiki/Largest_remainder_method

주제	출처	URL
Revelator 계약·요율 가이드(비교용)	Revelator	https://docs.revelator.com/pro/getting-started/onboarding-to-revelator/ingesting-your-rights
SoundCloud Fan-powered Royalties(비교용)	SoundCloud	https://help.soundcloud.com/hc/en-us/articles/1260801306810-Fan-powered-Royalties

출처 사용 원칙: 법령은 조문 링크, 표준은 공식 페이지, 수치는 공식 게시 페이지의 게시 시점 값. 2차 자료(Session Help Center, Wikipedia)는 개념 정의 확인용으로만 썼고 수치 근거로 쓰지 않았다. 국내 전송사용료 배분율·시장 규모 수치는 본 문서 작성 시 1차 출처를 직접 확인하지 않았으므로 본문에 싣지 않았다. v0.1 확인 출처 외 신규 수치는 추가하지 않았다.

부록 D. 변경 이력

버전	일자	변경
v0.1	2026-09-29	초안. 응용 프로파일의 프로토타입을 기준으로 코어·확장점·개념을 정리
v0.2	2026-09-29	**언어 표준 규격으로 지위를 고정**하고, 문제 제기 구조를 **기존 권리 관리 체계(집중관리·식별자/메타데이터·계약·ODRL 계열·정산 스크립트) 대비**로 재편. 특정 프로파일 인용·제품 어휘·구현 현황표를 규범 축에서 제거. 봉투·digest·형식 태그를 중립 식별자로 교체. 표면 문법과 IR 예시를 마스터 수준으로 재작성. 7장은 참조 아키텍처와 적합성 프로그램만 남김

v0.2가 v0.1에서 유지한 것: 1장 산업 출처와 수치, 2장의 P1–P8과 요구사항 표, 5장의 열 개 개념과 불변식, 6장의 층·타입·확장점·평가 계약, 부록 C.

[확인 필요] 항목 모음

1. 헤더: 영문 가칭 **MDRL**(Music Derivative Rights Language). 확정 필요. 대안 표기는 헤더에 두지 않는다.
2. 4.5 / Q1: 봉투 필드 `lang` 유지 여부, `profile` 필드 추가 여부.
3. 5.4 / Q3: `Scope` 타입·meet 연산 IR과 grant 문법의 확정 시점(v0.3 제안).
4. 5.7 / Q5: 위임 홉 상한 2의 상수/파라미터 여부.
5. 6.4 / Q3: 동의 범위를 grant에, 승인을 프로파일에 둘지.
6. 6.6 / Q4: 원장 문법의 언어 승격 여부.
7. 6.9(a) / Q2: 마스터 추상 인코딩을 코어가 받을지.